RL-DP-Algorithms

July 21, 2026

1 RL-DP Algorithms

1.1 Index

Classic Dynamic Programming

What Defines a Classic DP Problem?

Ex: TSP (Traveling Salesman Problem)

1. Held-Karp (Bottom-Up / Tabulation)
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
2. Held-Karp (Top-Down / Memoisation)
   · Introduction · Algorithm · Engineering Tricks · Pseudocode

Reinforcement Learning Dynamic Programming

What Defines an RL DP Problem?

RL Model-Based Methods

Introduction

1. Value Iteration
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
2. Policy Iteration
   · Introduction · Algorithm · Engineering Tricks · Pseudocode

RL Model-Free Methods

Introduction
On-Policy vs Off-Policy Algorithms
Full Episodes vs Step-wise Updates

1. Monte Carlo (First-Visit, On-Policy)
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
2. Temporal Difference — TD(0)
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
3. n-Step TD
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
4. SARSA (State-Action-Reward-State-Action)
   · Introduction · Algorithm · Engineering Tricks · Pseudocode
5. Q-Learning
   · Introduction · Algorithm · Engineering Tricks · Pseudocode

Deep RL — Value-Based

1. DQN Naive (Deep Q-Network, no tricks)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

2. DQN + Target Network (DQN+TN)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

3. DQN + Experience Replay (DQN+ER)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

Deep RL — Policy Gradient Methods

What Is the Vanilla Policy Gradient Loss?

Vanilla Policy Gradient Methods:

1. REINFORCE (Monte Carlo Policy Gradient)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

2. Actor-Critic (AC)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

3. A2C (Advantage Actor-Critic)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

4. A3C (Asynchronous Advantage Actor-Critic)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

Advanced Policy Optimization Methods:

1. DDPG (Deep Deterministic Policy Gradient)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

2. SAC (Soft Actor-Critic)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

3. PPO (Proximal Policy Optimization)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

4. CQL (Conservative Q-Learning)

· Introduction · Algorithm · Engineering Tricks · Pseudocode

1.2 Classic Dynamic Programming

1.2.1 What Defines a *Classic* DP Problem?

Classic **Dynamic Programming (DP)** solves a problem by breaking it into overlapping subproblems, optimizing each one *once*, and combining those stored sub-solutions to build the answer to the full problem. Every subproblem's optimal value is cached in a **DP table**, which guarantees the *exact* global optimum — but that table can grow **exponentially** with problem size, so these methods are often memory-hungry.

The traits that make a problem a *classic* DP problem are:

- **A fully known, static, (usually) deterministic structure.** The costs, transitions, and constraints are all given up front — nothing is sampled or estimated from experience.
- **No agent, no actions-over-time, no reward.** The goal is a single **combinatorial optimum** (a shortest tour, a longest common subsequence, an optimal matrix-chain order), *not* a rule for behaving in an environment.
- **Exact optimality in one shot.** Because the whole structure is known, the recursion returns the provably optimal answer directly, rather than an approximation refined over many iterations.

This is exactly the contrast with the **reinforcement-learning DP** of the next section. There, the "problem" is a *sequential decision process* — an MDP — in which an agent repeatedly chooses actions, the transitions are typically **stochastic**, and the objective is a **long-run expected return** instead of a one-shot combinatorial cost. Both families reuse the same Bellman idea of building on optimal sub-solutions, but **classic DP optimizes a fixed structure**, whereas **RL DP plans a *policy* under uncertainty**.

The following are two solution methods for one of the most famous classic DP problems — the **TSP** (Traveling Salesman Problem).

1.2.2 TSP (Traveling Salesman Problem)

Held-Karp (Bottom-Up / Tabulation) Method **Notation:** - n — total number of cities - $d[i][j]$ — travel cost / distance from city i to city j - S — a *subset* of cities $\{2, \dots, n\}$ (not the MDP state space; city 1 is the fixed start) - $C(S,\, j)$ — minimum-cost path: starts at city 1, visits every city in S exactly once, ends at $j \in S$ - OPT — optimal (minimum-cost) Hamiltonian tour

Introduction: Exact DP algorithm for the Traveling Salesman Problem (TSP). Solves in $O(n^2\, 2^n)$ time and $O(n\, 2^n)$ space — exponential, but far better than the naive $O(n!)$ brute-force. *Bottom-up variant:* fills the DP table iteratively from smallest subsets to largest.

Problem: Given n cities $\{1, 2, \dots, n\}$ and a distance matrix $d[i][j]$, find the shortest Hamiltonian cycle (visit every city exactly once and return to the start). Fix city 1 as the starting city without loss of generality.

State definition: $C(S, j)$ = minimum cost of a path that starts at city 1, visits every city in subset $S \subseteq \{2, \dots, n\}$, and ends at city $j \in S$.

Algorithm:

Input: number of cities n, travel cost matrix $d[i][j]$

1. **Base cases** — paths of length 1 from city 1 to each other city:
   - For each $l \in \{2, \dots, n\}$:
   $$C(\{l\},\, l) = d[1][l]$$
2. **Fill table bottom-up** — iterate over subsets of increasing size:
   - **For** $|S| = 2, 3, \dots, n{-}1$ *(outer loop — subset sizes)*:
     - **For each** subset $S \subseteq \{2, \dots, n\}$ with $|S|$ elements *(middle loop — subsets)*:
       - **For each** $l \in S$ *(inner loop — ending city)*:
       $$C(S,\, l) = \min_{k \in S \setminus \{l\}} [C(S \setminus \{l\},\, k) + d[k][l]]$$
       "Best way to reach l = best way to reach some predecessor k in $S \setminus \{l\}$, plus edge $k \to l$"
3. **Close the tour** — return to city 1:
   $$\text{OPT} = \min_{j \in \{2, \dots, n\}} [C(\{2, \dots, n\},\, j) + d[j][1]]$$
4. **Reconstruct path** by backtracking through stored argmin choices.

   Return optimal tour and cost OPT

Notes:

- **Bitmask encoding:** subsets S are represented as bitmasks of $n{-}1$ bits, e.g. $S = \{2, 4\}$ with $n = 5$ is encoded as $\texttt{0101}_2 = 5$. This allows $O(1)$ set operations (add, remove, membership) via bitwise operators.
- **Time complexity:** $O(n^2\, 2^n)$ — for each of 2^{n-1} subsets, we iterate over $O(n)$ ending cities and $O(n)$ predecessors.

- **Space complexity:** $O(n\,2^n)$ — the DP table $C[S][j]$ stores one value per (subset, ending city) pair.
- Practical limit: $n \approx 20$–25 cities (due to exponential memory/time).
- The bottom-up approach fills the entire table even if some entries are never needed for the final answer.

Additional Known Engineering Tricks

- **Bitmask subset encoding** — represent the visited set S as an $(n-1)$-bit integer; add/remove a city becomes `mask | (1<<i)` / `mask & ~(1<<i)`. *Why it helps:* set operations are $O(1)$ machine instructions instead of $O(n)$ container manipulations; the DP table can be a flat array indexed by `(mask, j)`. *Interaction:* prerequisite for every other low-level trick below — the pseudocode uses abstract set notation $S \subseteq \{2, \dots, n\}$; the bitmask is the standard implementation realisation.
- **Gosper's hack for fixed-cardinality enumeration** — given a mask with k bits, the next mask with k bits is `t = mask | (mask-1); next = (t+1) | (((~t & -~t)-1) >> (__ctz(mask)+1))`. *Why it helps:* iterates over all $\binom{n-1}{k}$ subsets of size k in constant time per step, instead of skipping over the other $2^{n-1} - \binom{n-1}{k}$ masks.
- **Flat 2D table** $C[\mathbf{mask} \cdot n + j]$ — store the DP values in a contiguous $2^{n-1} \cdot n$ array instead of a nested dict/2D-array-of-arrays. *Why it helps:* cache-friendly linear access pattern in the inner $\min_k$ loop; on modern hardware this is the single biggest constant-factor win.
- **Parent / predecessor table** — alongside $C[S][j]$, store the argmin k that achieved it. *Why it helps:* enables $O(n)$ path reconstruction by walking back through the parent pointers, avoiding a second DP pass.
- **Symmetry exploitation for symmetric distance matrices** — when $d[i][j] = d[j][i]$, the tour and its reverse have equal cost, so you may fix not just the start city but also the *direction* (e.g., require the second city to have the smallest index among $S \setminus \{1\}$). *Why it helps:* halves the state space and runtime constant factor.
- **Best-known-tour pruning** — compute a quick heuristic upper bound (nearest-neighbour or Christofides) before the DP, then skip any partial $C(S, j) + d[j][1]$ that already exceeds it. *Why it helps:* turns Held–Karp into a branch-and-bound variant that prunes large swaths of the table on structured instances.

Algorithm 1.3 Held–Karp (bottom-up / tabulation) — Raw Pseudocode (without additional tricks) *[Held & Karp, 1962]*

Here is the pseudocode for the Held–Karp (bottom-up / tabulation) core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: number of cities n, distance matrix $d[i][j]$
// Base cases — singleton subsets
for each $l \in \{2, \dots, n\}$ **do**
  $C(\{l\}, l) \leftarrow d[1][l]$
end for
// Build subsets bottom-up by size
for $s = 2$ **to** $n - 1$ **do**

for each $S \subseteq \{2, \dots, n\}$ with $|S| = s$ **do**
for each $l \in S$ **do**
$C(S, l) \leftarrow \min_{k \in S \setminus \{l\}} \{ C(S \setminus \{l\}, k) + d[k][l] \}$
end for
end for
end for
// Close the tour back to city 1
OPT $\leftarrow \min_{l \in \{2,\dots,n\}} \{ C(\{2, \dots, n\}, l) + d[l][1] \}$
Output: OPT

Algorithm 1.3 Held–Karp (bottom-up / tabulation) — Pseudocode with Additions (Engineering Tricks) *[Held & Karp, 1962]*

Here is the pseudocode for the Held–Karp (bottom-up / tabulation) algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: number of cities n, distance matrix $d[i][j]$
✦ Represent every subset $S \subseteq \{2, \dots, n\}$ as an $(n-1)$-bit integer ✦ (Bitmask subset encoding)
✦ Allocate flat table $C[\text{mask} \cdot n + j]$ of size $2^{n-1} \cdot n$ ✦ (Flat 2D table)
✦ Allocate parent table $P[\text{mask} \cdot n + j]$ for tour reconstruction ✦ (Parent / predecessor table)
✦ Compute heuristic upper bound UB (e.g., nearest-neighbour tour) ✦ (Best-known-tour pruning)
// Base cases — singleton subsets
for each $l \in \{2, \dots, n\}$ **do**
$C[1\texttt{<<}(l-2), l] \leftarrow d[1][l]$
✦ $P[1\texttt{<<}(l-2), l] \leftarrow 1$ ✦ (Parent / predecessor table)
end for
// Build subsets bottom-up by size
for $s = 2$ **to** $n - 1$ **do**
✦ **for each** mask with popcount(mask) $= s$ *(via Gosper's hack)* **do** ✦ (Gosper's hack for fixed-cardinality enumeration)
✦ *(For symmetric d: fix direction so second city $<$ last city in subset)* ✦ (Symmetry exploitation for symmetric distance matrices)
for each l with bit l set in mask **do**
$C[\text{mask}, l] \leftarrow \min_{k \in \text{mask} \setminus \{l\}} \{ C[\text{mask} \oplus 1\texttt{<<}(l-2), k] + d[k][l] \}$
✦ $P[\text{mask}, l] \leftarrow \arg\min_k(\cdots)$ ✦ (Parent / predecessor table)
✦ **if** $C[\text{mask}, l] + d[l][1] \geq$ UB **then skip** ✦ (Best-known-tour pruning)
end for
end for
end for
OPT $\leftarrow \min_{l \in \{2,\dots,n\}} \{ C[\text{full}, l] + d[l][1] \}$
✦ Reconstruct optimal tour by following $P[\cdot]$ backwards from $\arg\min$ ✦ (Parent / predecessor table)
Output: OPT and optimal tour

✦✦✦

- **Disadvantage:** recursive call overhead and potential stack depth issues for large n. Python's default recursion limit may need to be increased.
- **Bitmask encoding** is used identically: S is an integer bitmask, memo table is a dictionary or 2D array indexed by (bitmask, j).
- **Comparison:**

	Bottom-Up (Tabulation)	Top-Down (Memoisation)
Approach	Iterative; fills entire table	Recursive; fills on demand
Subproblems computed	All $O(n\,2^n)$ entries	Only reachable entries
Overhead	Loop overhead only	Recursion + memo lookup
Implementation	Nested loops over subset sizes	Recursive function + cache
Memory pattern	Sequential, cache-friendly	Scattered, less cache-friendly

Additional Known Engineering Tricks

- **`functools.lru_cache` / explicit hash memo** — wrap `solve(S, j)` with `@lru_cache(maxsize=None)` (or use a manual `dict` keyed by `(mask, j)`). *Why it helps:* one line gives you the memoisation that defines top-down DP, with proper hashing and eviction-free behaviour. *Interaction:* requires hashable keys — use `int` bitmasks (next bullet).
- **Integer bitmask keys instead of `frozenset`** — represent S as a Python `int` rather than a `frozenset`/`tuple`. *Why it helps:* `int` hashing and equality are far cheaper than for frozensets; lookup speed dominates Held–Karp top-down runtime.
- **Recursion-limit bump and stack-safe iteration** — call `sys.setrecursionlimit(10**6)` for $n \geq 20$, or convert `solve` to an iterative form with an explicit `deque`-based stack. *Why it helps:* default CPython recursion limit (1000) is exceeded around $n \approx 18$ in the worst case; without the bump the algorithm crashes silently as n grows.
- **Branch-and-bound with global upper bound** — keep a running best total-tour cost; inside `solve(S, j)`, if the partial cost plus a *lower bound* on the remaining $C(\bar{S}, \cdot)$ (e.g., sum of min outgoing edges) exceeds the best, return $+\infty$. *Why it helps:* turns memoisation into pruning — many subproblems are never expanded. *Interaction:* only helps when a good initial heuristic tour exists (e.g., nearest-neighbour upper bound).
- **Lazy memo (dict not array)** — store only the subproblems actually visited, not the full $O(n \cdot 2^n)$ table. *Why it helps:* on instances where large parts of the state space are unreachable, memory drops by orders of magnitude — the headline advantage over bottom-up tabulation.
- **Tail-shared subproblem detection** — observe that `solve(S, j)` and `solve(S, j')` share all `solve(S\{j}, k)` sub-calls; batch them in a single sweep over S at the end of each subproblem. *Why it helps:* reduces redundant memo lookups; mirrors what bottom-up gets for free.

Algorithm 1.4 Held–Karp (top-down / memoisation) — Raw Pseudocode (without additional tricks) *[Held & Karp, 1962]*

Here is the pseudocode for the Held–Karp (top-down / memoisation) core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: number of cities n, distance matrix $d[i][j]$
Initialize memo table $C[S][j] \leftarrow$ `None` for all (S, j)
function `solve`$(S,\ j)$:
  if $S = \{j\}$ **then return** $d[1][j]$
  if $C[S][j] \neq$ `None` **then return** $C[S][j]$
  $C[S][j] \leftarrow \min_{k \in S \setminus \{j\}} \{\, \texttt{solve}(S \setminus \{j\},\ k) + d[k][j] \,\}$
  return $C[S][j]$
OPT $\leftarrow \min_{j \in \{2,\ldots,n\}} \{\, \texttt{solve}(\{2,\ldots,n\},\ j) + d[j][1] \,\}$
Output: OPT

Algorithm 1.4 Held–Karp (top-down / memoisation) — Pseudocode with Additions (Engineering Tricks) *[Held & Karp, 1962]*

Here is the pseudocode for the Held–Karp (top-down / memoisation) algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: number of cities n, distance matrix $d[i][j]$
✦ `sys.setrecursionlimit`(10^6) *(or convert to iterative form with explicit stack)* ✦ (Recursion-limit bump and stack-safe iteration)
✦ Initialise memo as empty `dict`: $C = \{\}$ *(lazy — only visited subproblems stored)* ✦ (Lazy memo (dict not array))
✦ Compute heuristic upper bound UB (nearest-neighbour tour) ✦ (Branch-and-bound with global upper bound)
✦ `@lru_cache(maxsize=None)` ✦ (functools.lru_cache / explicit hash memo)
function `solve`(✦ mask ✦ (Integer bitmask keys instead of frozenset), j):
  if mask $=$ `1<<`$(j-2)$ **then return** $d[1][j]$
  ✦ `lb` $\leftarrow$ lower bound on remaining tour from (mask, j) ✦ (Branch-and-bound with global upper bound)
  ✦ **if** partial cost $+$`lb` $\geq$ UB **then return** $+\infty$ ✦ (Branch-and-bound with global upper bound)
  ✦ Batch all $k \in \text{mask} \setminus \{j\}$ sub-calls sharing the same sub-mask ✦ (Tail-shared subproblem detection)
  $C[\text{mask}][j] \leftarrow \min_{k \in \text{mask} \setminus \{j\}} \{\, \texttt{solve}(\text{mask} \oplus \texttt{1<<}(j-2),\ k) + d[k][j] \,\}$
  return $C[\text{mask}][j]$
OPT $\leftarrow \min_{j \in \{2,\ldots,n\}} \{\, \texttt{solve}(\texttt{full_mask},\ j) + d[j][1] \,\}$
Output: OPT

✦✦✦

1.3 Reinforcement Learning Dynamic Programming

1.3.1 What Defines an *RL* DP Problem?

Reinforcement-learning Dynamic Programming applies the same Bellman "reuse optimal sub-solutions" principle, but to a fundamentally different kind of problem: a **Markov Decision Process (MDP)**. A problem here is described by *states*, *actions*, a *transition model* $P(s' \mid s, a)$, and a *reward* $r(s, a, s')$ — and the goal is not a single combinatorial answer but an **optimal policy** π^*: a rule telling the agent which action to take in every state so as to maximize its **long-run expected (discounted) return**.

The traits that make a problem an *RL* DP problem are:

- **A sequential decision process.** An agent acts over time; each action shapes future states and rewards, so the objective spans an entire trajectory rather than a static structure.
- **Stochastic transitions and expected value.** The next state is generally random, so the Bellman update takes an **expectation** $\sum_{s'} P(s' \mid s, a)\,[\cdots]$ instead of a deterministic combination of subproblems.
- **A known model, solved by *planning*.** The *model-based* methods here — **Value Iteration** and **Policy Iteration** — assume P and r are known and **iterate the Bellman equations to convergence**, computing value functions and improving the policy. (Learning these from *sampled* experience, with no model, is the *model-free* territory of the following chapter.)

In short: *classic* DP optimizes a fully specified, static combinatorial structure and returns an exact optimum; *RL* DP plans an optimal **policy** for acting under uncertainty in an MDP. The machinery (Bellman recursion, tabular sub-solutions) is shared — what changes is the **problem itself**: a one-shot optimization versus an ongoing, stochastic decision process.

1.4 RL Model-Based Methods

1.4.1 Introduction

Model-based methods assume the environment's dynamics are **fully known** — the transition probabilities $P(s' \mid s, a)$ and the reward function $r(s, a)$ are given up front. Rather than *learning* from sampled experience, the agent ***plans***: it iterates the Bellman equations to convergence to compute the value functions and improve the policy.

· **Requires a model** — needs the complete MDP (P, r) before any computation begins.

· **Planning, not sampling** — updates sweep over *all* states using the known dynamics; no interaction with the environment is needed.

· **Exact but limited** — converges to the true optimal values, yet only tractable when the state space is small enough to enumerate. *Examples: Value Iteration, Policy Iteration.*

1.4.2 Value Iteration:

Notation: - $r(s, a, s')$ — reward received when transitioning from s to s' via action a - v — temporary copy of $V(s)$ before the Bellman update, used to measure change Δ

Introduction: Solves an MDP by iteratively applying the Bellman optimality update $V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a)[r(s, a, s') + \gamma V(s')]$ until V converges, then reads off the greedy policy. *Idea:* fuse policy evaluation and policy improvement into a single max-backup — no need to fully evaluate each intermediate policy as Policy Iteration does.

Algorithm:

Input: MDP (S, A, P, r, γ), threshold $\theta > 0$

1. Initialize $V(s)$ arbitrarily for all $s \in S$ (e.g., $V(s) = 0$)
2. **Repeat** *(outer loop — sweeps)*:
   - $\Delta \leftarrow 0$
   - For each $s \in S$ *(inner loop — states)*:
     - $v \leftarrow V(s)$
     - $V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V(s')\right]$
     - $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
   - Until $\Delta < \theta$ *(convergence check)*
3. **Output policy** — for each $s \in S$:

$$\pi(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V(s')\right]$$

Return π

Additional Known Engineering Tricks

- **In-place (Gauss–Seidel) updates** — overwrite $V(s)$ as soon as the new value is computed within a sweep, rather than building a fresh copy from V_k. *Why it helps:* the freshest values are reused immediately by later states in the same sweep, so the effective contraction is tighter and convergence typically needs fewer sweeps. *Interaction:* now the *order* of states matters — pairs naturally with **prioritized sweeping** and **asynchronous DP**, and is mutually exclusive with the *vectorised* synchronous backup below.
- **Prioritized sweeping** *(Moore & Atkeson, 1993)* — maintain a priority queue keyed by each state's current Bellman residual; on every step, pop the highest-error state, update it, and re-queue its predecessors. *Why it helps:* compute is concentrated where V is still changing significantly, instead of wasting full sweeps on already-converged states. *Interaction:* assumes in-place updates; in the limit it is the most aggressive scheduling of asynchronous DP.
- **Asynchronous DP** — update only a subset of states per pass (random, trajectory-based, or reachable-from-start). *Why it helps:* avoids touching irrelevant states in large MDPs where most states are unreachable from typical start distributions, and parallelises across processors without coordination. *Interaction:* generalises in-place sweeping; convergence still holds as long as every state is visited infinitely often.

- **Vectorised Bellman backup** — store P as a dense $(|S|, |A|, |S'|)$ tensor and compute one synchronous sweep via `einsum` / matrix products. *Why it helps:* turns the triple loop into a single BLAS call, giving order-of-magnitude wall-clock speed-ups on dense MDPs. *Interaction:* implements the *synchronous* Jacobi-style backup — incompatible with strict in-place updates; pick one or the other depending on whether speed or sweep count matters more.
- **Span-seminorm convergence test** — replace $\Delta < \theta$ with $\mathrm{sp}(V_{k+1} - V_k) < \theta\,(1-\gamma)/\gamma$, where $\mathrm{sp}(x) = \max x - \min x$. *Why it helps:* yields a tight ε-optimal-*policy* guarantee from the contraction bound, so the loop stops as soon as the policy is provably near-optimal rather than at an arbitrary max-norm threshold.
- **Action-set pruning / dominance** — at each state, drop actions that are dominated (lower $Q(s,a)$ than another action under all admissible V bounds). *Why it helps:* shrinks the inner $\max_a$ in every sweep — important when $|A|$ is large or many actions are clearly suboptimal early.

Algorithm 1.1 Value iteration — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §4.4]*

Here is the pseudocode for the value iteration core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: MDP (S, A, P, r, γ), threshold $\theta > 0$
Initialize $V(s) \leftarrow 0$ for all $s \in S$
repeat
  $\Delta \leftarrow 0$
  for each $s \in S$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a)\,[\,r(s,a,s') + \gamma\, V(s')\,]$
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
  end for
until $\Delta < \theta$
Output: deterministic policy $\pi(s) \leftarrow \arg\max_a \sum_{s'} P(s' \mid s, a)\,[\,r(s,a,s') + \gamma\, V(s')\,]$

Algorithm 1.1 Value iteration — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §4.4]*

Here is the pseudocode for the value iteration algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: MDP (S, A, P, r, γ), threshold $\theta > 0$
Initialize $V(s) \leftarrow 0$ for all $s \in S$
✦ Initialize priority queue PQ; insert every s with priority ∞ ✦ (Prioritized sweeping)
✦ Pre-compute predecessor sets pred(s) for all s ✦ (Prioritized sweeping)
repeat
  $\Delta \leftarrow 0$

✦ **while** PQ is not empty **do** *(update only high-residual states; or process a random / trajectory-based subset)* ✦ (Prioritized sweeping / Asynchronous DP)
✦ $s \leftarrow$ PQ.pop_max() ✦ (Prioritized sweeping)
$v \leftarrow V(s)$
✦ $A'(s) \leftarrow \{a \in A \mid a \text{ not dominated}\}$ ✦ (Action-set pruning / dominance)
✦ $V(s) \leftarrow \max_{a \in A'(s)} \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V(s') \right]$ *(overwrite in-place)* ✦ (In-place (Gauss–Seidel) updates)
$\Delta \leftarrow \max(\Delta, |v - V(s)|)$
✦ **for each** $s_p \in \text{pred}(s)$ **do** update PQ(s_p) with new residual **end for** ✦ (Prioritized sweeping)
✦ **end while** ✦ (Prioritized sweeping)
✦ **until** $\text{sp}(V_{k+1} - V_k) < \theta\,(1-\gamma)/\gamma$ *(span-seminorm test)* ✦ (Span-seminorm convergence test)
✦ *Alternative: replace the per-state loop with* $V \leftarrow$ *vectorised* $\max_a [P_a\,(r_a + \gamma V)]$ *via einsum (synchronous Jacobi-style; incompatible with in-place)* ✦ (Vectorised Bellman backup)
Output: $\pi(s) \leftarrow \arg\max_{a \in A'(s)} \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V(s') \right]$

✦✦✦

1.4.3 Policy Iteration

Notation: - $r(s, a, s')$ — reward on transition $s \xrightarrow{a} s'$ - v — temporary copy of $V(s)$ before update - policy_stable — flag that becomes false whenever any state's action changes during improvement

Introduction: Solves an MDP by alternating two steps until the policy stops changing: (i) **policy evaluation** — compute V^π for the current π by repeated Bellman expectation updates, and (ii) **policy improvement** — set $\pi(s) \leftarrow \arg\max_a \sum_{s'} P(s' \mid s, a)\big[r + \gamma\, V^\pi(s')\big]$. *Idea:* each improvement is guaranteed to produce a strictly better (or equal) policy (policy improvement theorem), so the algorithm converges in *finitely* many outer iterations on a finite MDP — typically far fewer than Value Iteration needs sweeps, at the cost of a full evaluation sweep per outer iteration.

Algorithm:

Input: MDP (S, A, P, r, γ), threshold $\theta > 0$

1. Initialize $\pi(s)$ arbitrarily for all $s \in S$
2. **Repeat** *(outer loop — policy iterations)*:
   - **— Policy Evaluation —**
   - **Repeat** *(middle loop — evaluation sweeps)*:
     - $\Delta \leftarrow 0$
     - For each $s \in S$ *(inner loop — states)*:
       - $v \leftarrow V(s)$
       - $V(s) \leftarrow \sum_{s'} P(s' \mid s, \pi(s))\,\big[r(s, \pi(s), s') + \gamma\, V(s')\big]$
       - $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
     - Until $\Delta < \theta$ *(evaluation convergence check)*
   - **— Policy Improvement —**
   - policy_stable $\leftarrow$ *true*
   - For each $s \in S$ *(single pass — states)*:
     - old_action $\leftarrow \pi(s)$
     - $\pi(s) \leftarrow \arg\max_a \sum_{s'} P(s' \mid s, a)\,\big[r(s, a, s') + \gamma\, V(s')\big]$
     - If old_action $\neq \pi(s)$: policy_stable $\leftarrow$ *false*

   Until policy_stable = *true (outer convergence check)*
3. Return π, V

Additional Known Engineering Tricks

- **Modified Policy Iteration (MPI)** *(Puterman & Shin, 1978)* — replace exact policy evaluation with just k Bellman-expectation sweeps (typically $k \in [5, 50]$). *Why it helps:* full evaluation is wasted effort early on when the policy is still bad and will change anyway — partial evaluation is enough to drive improvement, so total work drops dramatically. *Interaction:* defines the spectrum from value iteration ($k = 1$) to classical policy iteration ($k = \infty$).
- **Generalised Policy Iteration (GPI)** — interleave evaluation and improvement at *any* granularity (one state at a time, asynchronously, even action-selection-driven). *Why it helps:*

makes the algorithm an anytime procedure that converges as long as both processes make progress on every state infinitely often. *Interaction:* MPI is the special case where each interleaved evaluation phase is k full sweeps; pairs with **asynchronous policy evaluation**.

- **Warm-start V across outer iterations** — keep V between policy improvements rather than re-initialising to zero. *Why it helps:* a newly-improved policy's value function is usually close to the previous one, so inner evaluation needs only a few sweeps to re-converge. *Interaction:* this is precisely what makes MPI's small k work — without warm-start, truncating evaluation would forget all prior progress.
- **Action-value caching during improvement** — during the improvement sweep, compute $Q(s,a) = \sum_{s'} P(s' \mid s,a)[r + \gamma V(s')]$ once per (s,a), store it, then take arg max and the change-flag from the same table. *Why it helps:* the improvement step otherwise recomputes the same sum twice — once for the argmax, once when checking if anything changed.
- **Policy-stable early exit** — break the inner evaluation as soon as no state's greedy action would change. *Why it helps:* once the policy has stabilised, further evaluation cannot improve it; the outer loop will terminate next round anyway.
- **Bertsekas's λ-policy-iteration** — bootstrap evaluation with λ-returns instead of one-step Bellman backups. *Why it helps:* trades a controllable amount of bias for much faster propagation of value information across long horizons. *Interaction:* generalises both MPI and value iteration as endpoints.

Algorithm 1.2 Policy iteration — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §4.3]*

Here is the pseudocode for the policy iteration core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: MDP (S, A, P, r, γ), threshold $\theta > 0$
Initialize $V(s) \in \mathbb{R}$ and $\pi(s) \in A$ arbitrarily for all $s \in S$
repeat
  // — Policy Evaluation —
  repeat
    $\Delta \leftarrow 0$
    for each $s \in S$ **do**
      $v \leftarrow V(s)$
      $V(s) \leftarrow \sum_{s'} P(s' \mid s, \pi(s)) \left[r(s, \pi(s), s') + \gamma\, V(s') \right]$
      $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
    end for
  until $\Delta < \theta$
  // — Policy Improvement —
  policy_stable $\leftarrow$ **true**
  for each $s \in S$ **do**
    $a_{\text{old}} \leftarrow \pi(s)$
    $\pi(s) \leftarrow \arg\max_a \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma\, V(s') \right]$
    if $a_{\text{old}} \neq \pi(s)$ **then** policy_stable $\leftarrow$ **false**
  end for

until policy_stable = **true**
Output: $V \approx v_*$ and $\pi \approx \pi_*$

Algorithm 1.2 Policy iteration — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §4.3]*

Here is the pseudocode for the policy iteration algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: MDP (S, A, P, r, γ), threshold $\theta > 0$, ✦ evaluation sweeps k ✦ (Modified Policy Iteration (MPI))
Initialize $V(s) \in \mathbb{R}$ and $\pi(s) \in A$ arbitrarily for all $s \in S$
repeat
// — Policy Evaluation (truncated) —
✦ **for** sweep $= 1, \ldots, k$ **do** *(only k sweeps instead of until convergence)* ✦ (Modified Policy Iteration (MPI))
for each $s \in S$ **do**
$V(s) \leftarrow \sum_{s'} P(s' \mid s, \pi(s)) \left[r(s, \pi(s), s') + \gamma V(s') \right]$
end for
✦ **if** no state's greedy action would change **then break** *(early exit)* ✦ (Policy-stable early exit)
✦ **end for** *(warm-start: V is carried forward across outer iterations, not re-initialised)* ✦ (Warm-start V across outer iterations)
// — Policy Improvement —
policy_stable $\leftarrow$ **true**
for each $s \in S$ **do**
$a_{\text{old}} \leftarrow \pi(s)$
✦ Cache $Q(s, a) \leftarrow \sum_{s'} P(s' \mid s, a)[r + \gamma V(s')]$ for all a *(compute once, reuse for argmax and stability check)* ✦ (Action-value caching during improvement)
✦ $\pi(s) \leftarrow \arg\max_a Q(s, a)$ ✦ (Action-value caching during improvement)
if $a_{\text{old}} \neq \pi(s)$ **then** policy_stable $\leftarrow$ **false**
end for
✦ *(Evaluation and improvement may be interleaved at any granularity — per-state, per-action — under the GPI framework)* ✦ (Generalised Policy Iteration (GPI))
until policy_stable = **true**
Output: $V \approx v_*$ and $\pi \approx \pi_*$

✦✦✦

✦ ✦ ✦

1.5 RL Model-Free Methods

✦ ✦ ✦

1.5.1 Introduction

Model-free methods drop the assumption that the dynamics are known — $P(s' \mid s, a)$ and $r(s, a)$ are **unavailable**. Instead, the agent ***learns*** value functions and/or policies directly from **sampled transitions** (s, a, r, s') gathered by interacting with the environment.

· **No model needed** — the dynamics are never written down; everything is estimated from experience.

· **Learning by sampling** — updates use individual sampled transitions (or whole episodes), so they scale to large or unknown environments where enumeration is impossible.

· **General but noisier** — the workhorse of modern (deep) RL, at the cost of sampling variance. *Examples: Monte Carlo, TD, SARSA, Q-learning, and every policy-gradient method below.*

	Model-Based	Model-Free
Dynamics P, r	Known up front	Unknown — sampled
Core operation	Planning (Bellman sweeps)	Learning from experience
Environment interaction	Not required	Required
Scales to large / unknown MDPs	No (needs enumeration)	Yes
Examples	Value & Policy Iteration	MC, TD, SARSA, Q-learning, PG methods

1.5.2 On-Policy vs Off-Policy Algorithms

Model-free control methods learn Q from sampled transitions, but they differ in *which* policy generates those samples vs. *which* policy is being improved.

- **Behavior policy** μ — the policy actually used to act in the environment (generates the data).
- **Target policy** π — the policy whose value we are learning / trying to improve.

· **On-policy:** $\mu = \pi$. The agent learns about the same policy it follows, including its exploration. *Examples: MC control, SARSA, REINFORCE.*

· **Off-policy:** $\mu \neq \pi$. **The agent follows an exploratory behavior policy** μ but learns about a different (often greedy) target policy π. *Examples: Q-learning, DQN.*

	On-Policy	Off-Policy
Data source	Current policy only	Any policy (incl. old / human / replay)
TD target uses	Action sampled from π (e.g. $Q(s', a')$)	Action chosen by target π (e.g. $\max_{a'} Q(s', a')$)
Sample efficiency	Lower — old data becomes stale	Higher — can reuse past experience (replay)
Exploration cost	Reflected in learned values (safer)	Ignored in target (can be riskier)
Stability	Generally more stable	Can diverge with function approximation

More elaborate detail about off-policy vs on-policy Although the definitions are told above, later you might be confused by some upcoming materials. Here is a more elaborate disambiguation:

There are actually 2 definitions for Off-policy/On-policy methods:

- **Definition 1 (strict / Sutton & Barto)** On-policy: the policy generating data = the policy being evaluated/improved. Off-policy: they differ. Under this strict definition, yes — PPO with K>1 epochs is technically off-policy starting at epoch 1, because $\pi_\theta \neq \pi_$old. And the importance ratio $\pi_\theta/\pi_$old is literally an importance sampling correction, which is the textbook off-policy tool. By this definition, A2C with K=1 is on-policy (one update, then throw data away), but A2C with K>1 would also become technically off-policy — just like PPO.
- **Definition 2 (practical / common usage)** On-policy: uses only recently collected data, discards it after a few updates, relies on importance sampling at most for nearby distributions. Off-policy: uses a replay buffer with arbitrarily old data, learns a value function that doesn't depend on the behavior policy (Q-learning). Under this practical definition, PPO and A2C are on-policy because the data is essentially fresh and the policies are kept close. DQN and SAC are off-policy because they reuse data from policies arbitrarily far in the past.

Under this practical definition, PPO and A2C are on-policy because the data is essentially fresh and the policies are kept close. DQN and SAC are off-policy because they reuse data from policies arbitrarily far in the past.

Summary Off-policy/On-policy table:

Algorithm	Off-policy	On-policy	Reason
Monte Carlo control	✓	✓	Standard MC control learns from episodes of the policy being improved; an off-policy variant exists via importance sampling.**Mostly known for:** On-policy
SARSA	✗	✓	TD target uses the next action actually taken by the current policy ($Q(s', a')$).
Expected SARSA	✓	✓	Target averages $Q(s', \cdot)$ over the target policy; on-policy when target = behavior, off-policy when they differ (greedy target $\Rightarrow$ Q-learning).**Mostly known for:** On-policy
n-step SARSA	✗	✓	Multi-step return bootstrapped from actions taken by the current policy.
Q-learning	✓	✗	Target uses $\max_{a'} Q(s', a')$ (greedy π), independent of the exploratory behavior policy μ.
DQN	✓	✗	Q-learning with a replay buffer — learns the greedy policy from arbitrarily old transitions.
REINFORCE	✗	✓	Policy gradient estimated from trajectories sampled by the current policy.

Algorithm	Off-policy	On-policy	Reason
A2C / A3C	✗	✓	Actor-critic updates computed from fresh rollouts of the current policy.
TRPO	✓	✓	Uses importance sampling within a trust region, so technically off-policy, but data is fresh and kept near-policy.**Mostly known for:** On-policy
PPO	✓	✓	With $K > 1$ epochs it reuses data via a clipped importance ratio (technically off-policy), but keeps π close to π_{old}.**Mostly known for:** On-policy
DDPG	✓	✗	Deterministic policy gradient trained off a replay buffer.
TD3	✓	✗	DDPG with twin critics and target-policy smoothing; still replay-buffer based.
SAC	✓	✗	Maximum-entropy actor-critic trained from a replay buffer.

✦✦✦

1.5.3 Full Episodes vs Step-wise Updates (Algorithms' different behavior)

Algorithms differ in *how much* of a trajectory they observe before performing each update — anywhere from a single transition to a full episode.

· **Step-wise (bootstrapping)** — update after each step using the current value estimate. Lower variance, biased by the current estimate.

· **Fixed-length rollouts** — collect n steps and then update; sits between the two extremes.

· **Complete episodes** — wait until the trajectory terminates before updating. Lower bias (true returns), higher variance.

Algorithm	Updates from
Monte Carlo	Complete episodes
TD(0)	Each step (bootstraps)
n-step TD	n-step rollouts (full episodes only if $n = \infty$)
SARSA	Each step (bootstraps)
Q-learning	Each step (bootstraps)
REINFORCE	Complete episodes
PPO	Fixed-length rollouts (no need for full episodes)

✦✦✦

1.5.4 Monte Carlo (First-Visit, On-Policy)

Model-free, on-policy, learns from ***complete*** *episodes.*

Notation: - G — cumulative discounted return (accumulated backwards from episode end) - T — terminal time step (total episode length) - Returns(s, a) — list of all observed returns for pair (s, a) across all episodes so far

Introduction: Model-free, on-policy control that learns $Q(s, a)$ from sample returns of ***complete*** episodes (no bootstrapping). *Idea:* run the current ε-greedy policy, record the actual discounted return G_t from each (first-visit) state–action pair, and use the running average across episodes as the estimate $Q(s, a)$; then improve π to be ε-greedy w.r.t. Q. *Why it helps:* unbiased — uses real returns rather than bootstrapped estimates — at the cost of high variance and needing episode termination.

Algorithm:

Input: number of episodes n, discount γ, exploration rate ε

1. Initialize $Q(s, a)$ arbitrarily for all $s \in S,\ a \in A$; Returns$(s, a) \leftarrow \emptyset$; $\pi \leftarrow \varepsilon$-greedy w.r.t. Q
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - **— Generate Episode —**
   - Generate a full episode using π: $s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$ (until terminal)
   - **— Compute Returns and Update —**
   - $G \leftarrow 0$
   - **For** $t = T{-}1, T{-}2, \ldots, 0$ *(inner loop — steps, backwards)*:
     - $G \leftarrow \gamma G + r_{t+1}$
     - If $(s_t, a_t) \notin \{(s_0, a_0), \ldots, (s_{t-1}, a_{t-1})\}$ *(first-visit check)*:
       - Returns(s_t, a_t) += G
       - $Q(s_t, a_t) \leftarrow$ mean(Returns(s_t, a_t))
       - Update $\pi(s_t)$ to be ε-greedy w.r.t. $Q(s_t, \cdot)$:

$$\pi(a \mid s_t) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|A|} & \text{if } a = \arg\max_{a'} Q(s_t, a') \\ \dfrac{\varepsilon}{|A|} & \text{otherwise} \end{cases}$$

3. Return Q, π

✦ ✦ ✦

Notes:

- Must wait until episode **ends** to learn (no bootstrapping).
- G is the actual discounted return from step t to the end of the episode.
- *First-visit*: only update $Q(s, a)$ using the return from the **first** time (s, a) is visited in an episode. *Every-visit* MC updates for every occurrence.

Additional Known Engineering Tricks

- **Incremental mean update** — replace `Returns(s,a).append(G); Q ← mean(Returns)` with $Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}(G - Q(s,a))$. *Why it helps:* removes the unbounded per-pair return list — constant memory per (s,a) pair and constant-time updates instead of $O(N)$ averaging. *Interaction:* mathematically identical to the literal mean shown in the pseudocode; turning $\alpha = 1/N(s,a)$ into a *constant* α gives an exponentially-weighted moving average (non-stationary-friendly).
- **Exploring starts** *(Sutton & Barto, §5.3)* — start each episode at a uniformly random (s_0, a_0) pair rather than the fixed initial state. *Why it helps:* guarantees every (s,a) is sampled with positive probability under any policy, including deterministic greedy ones; otherwise on-policy MC can starve some pairs and never improve them. *Interaction:* an alternative to ε-greedy that allows learning a *deterministic* optimal policy directly; the two tricks are usually mutually exclusive.
- **GLIE schedule** *(Greedy in the Limit with Infinite Exploration)* — anneal $\varepsilon_k = 1/k$ (or c/k) with the episode count so that exploration vanishes in the limit but every action is still tried infinitely often. *Why it helps:* gives an *asymptotic* convergence guarantee to π^* for on-policy MC control, which fixed ε does not.
- **Every-visit MC** — update on *every* occurrence of (s,a) in the episode, not only the first. *Why it helps:* lower variance estimator (more samples per episode) at the cost of a small bias that vanishes asymptotically. *Interaction:* the trick is binary with first-visit (you pick one); first-visit is preferred for theoretical clarity, every-visit for raw sample efficiency.
- **Weighted importance sampling (off-policy MC)** — collect episodes under a behaviour policy μ, weight each return by $W = \prod_t \pi(a_t \mid s_t)/\mu(a_t \mid s_t)$, then use $Q \leftarrow \sum WG / \sum W$. *Why it helps:* lets you evaluate / improve a target π from data generated by any (sufficiently exploratory) μ; weighted form is biased but has bounded variance unlike ordinary IS.

Algorithm 2.1 First-visit Monte Carlo control (on-policy, ε-greedy) — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §5.4]*

Here is the pseudocode for the first-visit Monte Carlo control core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: number of episodes n, discount γ, exploration rate ε
Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A$
Returns$(s,a) \leftarrow \emptyset$ for all s, a
$\pi \leftarrow \varepsilon$-greedy w.r.t. Q
for episode $= 1, 2, \ldots, n$ **do**
  Generate an episode using π: $s_0, a_0, r_1,\ s_1, a_1, r_2,\ \ldots,\ s_{T-1}, a_{T-1}, r_T$
  $G \leftarrow 0$
  for $t = T-1$ **down to** 0 **do**
    $G \leftarrow \gamma\, G + r_{t+1}$
    if (s_t, a_t) does not appear in $(s_0, a_0), \ldots, (s_{t-1}, a_{t-1})$ **then**
      Append G to Returns(s_t, a_t)
      $Q(s_t, a_t) \leftarrow$ average(Returns(s_t, a_t))
      $\pi(\cdot \mid s_t) \leftarrow \varepsilon$-greedy w.r.t. $Q(s_t, \cdot)$
    end if

end for
end for
Output: $Q \approx q_*$, $\pi \approx \pi_*$

Algorithm 2.1 First-visit Monte Carlo control (on-policy, ε-greedy) — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §5.4]*

Here is the pseudocode for the first-visit Monte Carlo control algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: number of episodes n, discount γ, ✦ initial ε_1 ✦ (GLIE schedule)
Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A$
✦ Initialise $N(s,a) \leftarrow 0$ for all s, a *(visit counts for incremental update)* ✦ (Incremental mean update)
$\pi \leftarrow \varepsilon$-greedy w.r.t. Q
for episode $= 1, 2, \dots, n$ **do**
✦ $\varepsilon \leftarrow 1/\text{episode}$ *(anneal toward greedy — GLIE)* ✦ (GLIE schedule)
✦ Start episode at uniformly random (s_0, a_0) *(alternative to ε-greedy)* ✦ (Exploring starts)
Generate an episode using π: $s_0, a_0, r_1,\ s_1, a_1, r_2,\ \dots,\ s_{T-1}, a_{T-1}, r_T$
$G \leftarrow 0$
for $t = T-1$ **down to** 0 **do**
$G \leftarrow \gamma G + r_{t+1}$
✦ *(Every-visit variant: skip the first-visit check and update on every occurrence)* ✦ (Every-visit MC)
if (s_t, a_t) does not appear in $(s_0, a_0), \dots, (s_{t-1}, a_{t-1})$ **then**
✦ $N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$ ✦ (Incremental mean update)
✦ $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{N(s_t, a_t)}(G - Q(s_t, a_t))$ *(replaces list-based mean)* ✦ (Incremental mean update)
$\pi(\cdot \mid s_t) \leftarrow \varepsilon$-greedy w.r.t. $Q(s_t, \cdot)$
end if
end for
✦ *(Off-policy variant: weight returns by $W = \prod_t \pi/\mu$, use $Q \leftarrow \sum WG / \sum W$)* ✦ (Weighted importance sampling (off-policy MC))
end for
Output: $Q \approx q_*$, $\pi \approx \pi_*$

✦✦✦

1.5.5 Temporal Difference — TD(0)

*Model-free; bootstraps after **each step** (no need to wait for episode end).*

Notation: - $\alpha \in (0, 1]$ — step size (learning rate) - s' — next state reached after taking action a from state s

Introduction: Model-free policy-evaluation method that estimates V^π online by bootstrapping after ***each step*** (no need to wait for episode end). *Idea:* update $V(s) \leftarrow V(s)+\alpha[r+\gamma V(s')-V(s)]$, combining a single sampled reward with the current estimate of future value. *Why it helps over Monte Carlo:* learns online from incomplete episodes and has much lower variance, at the price of bias from the bootstrapped target.

Algorithm:

Input: policy π, step size $\alpha \in (0, 1]$, discount γ, number of episodes n

1. Initialize $V(s)$ arbitrarily for all $s \in S$; $\quad V(\text{terminal}) = 0$
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - Initialize s (starting state)
   - **Repeat** for each step *(inner loop — time steps)*:
     - $a \leftarrow \pi(s)$
     - Take action a, observe $r,\ s'$
     - $V(s) \leftarrow V(s) + \alpha \underbrace{\Big[r + \gamma V(s') - V(s)\Big]}_{\delta \text{ (TD error)}}$ *(TD update)*
     - $s \leftarrow s'$
   - Until s' is terminal
3. Return V

✦ ✦ ✦

Notes:

- **TD target** $= r+\gamma V(s')$: a *biased* estimate of the true return, since $V(s')$ is itself an estimate — this is **bootstrapping**.
- **TD error** $\delta = r + \gamma V(s') - V(s)$: measures surprise — how much better the observed transition was vs. expectation.
- Unlike MC, TD can learn **before** the episode ends (online learning).
- Unlike DP, TD does **not** need the model $P(s' \mid s, a)$ — it samples transitions.
- This is *prediction only* (evaluating a given π). For control, see SARSA / Q-learning.

Additional Known Engineering Tricks

- **Eligibility traces — TD(λ)** *(Sutton, 1988)* — maintain a per-state trace $e(s)$ that decays with $\gamma\lambda$, and update *all* states at every step: $V(s)$ += $\alpha\delta\, e(s)$. *Why it helps:* propagates the TD error backwards through the recent trajectory in one shot, so credit reaches earlier states without waiting n steps. *Interaction:* recovers TD(0) at $\lambda = 0$ and Monte Carlo at $\lambda = 1$ — bridges TD(0) and MC algorithmically, not just statistically.
- **Robbins–Monro step-size schedule** — choose α_t satisfying $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$, e.g. $\alpha_t = 1/(1+\text{visits}(s)/c)$ per-state. *Why it helps:* gives the formal almost-sure convergence guarantee for tabular TD(0); constant α converges only to a noise ball, not the fixed point.

- **Optimistic initial values** *(Sutton & Barto, §2.6)* — initialise $V(s)$ to a value strictly above the realistic return (e.g., $V(s) = R_{\max}/(1-\gamma)$). *Why it helps:* every visited state will *decrease* its value relative to neighbours, so unvisited states look better and the policy is implicitly driven to explore — no ε needed for evaluation under a greedy policy.
- **Gradient-TD (GTD/TDC)** *(Sutton et al., 2009)* — when combining TD(0) with linear function approximation under off-policy data, replace the semi-gradient with a true gradient of the projected Bellman error. *Why it helps:* fixes the *deadly triad* (off-policy + bootstrapping + function approximation) divergence problem; standard semi-gradient TD can diverge, GTD provably converges.
- **Linear semi-gradient TD(0)** — replace the tabular $V(s)$ with $\hat{v}(s;\mathbf{w}) = \mathbf{w}^\top \phi(s)$, update $\mathbf{w} \mathrel{+}= \alpha\delta\phi(s)$. *Why it helps:* scales TD to large/continuous state spaces by sharing parameters across states; this is the gateway from tabular RL to deep RL (where it becomes a neural net, see DQN).
- **Mini-batch / replay TD prediction** — store recent (s, r, s') tuples in a buffer and sample mini-batches for offline TD updates. *Why it helps:* decorrelates samples for stochastic gradient methods and lets each transition be reused multiple times — same idea that powers DQN's experience replay, but applied to prediction.

Algorithm 2.2 Tabular TD(0) for estimating v_π — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §6.1]*

Here is the pseudocode for the tabular TD(0) core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: policy π, step size $\alpha \in (0, 1]$, discount γ, episodes n
Initialize $V(s)$ arbitrarily for all $s \in S$; $V(\text{terminal}) \leftarrow 0$
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  repeat (for each step of episode)
    $a \leftarrow$ action given by π for s
    Take action a; observe r, s'
    $V(s) \leftarrow V(s) + \alpha\,[\,r + \gamma\, V(s') - V(s)\,]$
    $s \leftarrow s'$
  until s is terminal
end for
Output: $V \approx v_\pi$

Algorithm 2.2 Tabular TD(0) for estimating v_π — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §6.1]*

Here is the pseudocode for the tabular TD(0) algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: policy π, ✦ initial step size α_0 ✦ (Robbins–Monro step-size schedule), discount γ, ✦ trace decay λ ✦ (Eligibility traces — TD(λ)), episodes n

✦ Initialise $V(s) \leftarrow R_{\max}/(1-\gamma)$ for all s *(optimistic)* ✦ (Optimistic initial values)
$V(\text{terminal}) \leftarrow 0$
✦ *(Alternative: replace tabular $V(s)$ with $\hat{v}(s;\mathbf{w}) = \mathbf{w}^\top \phi(s)$ — linear semi-gradient TD)* ✦ (Linear semi-gradient TD(0))
for episode $= 1, 2, \ldots, n$ **do**
  Initialise s
  ✦ $e(s) \leftarrow 0$ for all s *(eligibility traces)* ✦ (Eligibility traces — TD(λ))
  repeat (for each step of episode)
    $a \leftarrow$ action given by π for s
    Take action a; observe $r,\ s'$
    $\delta \leftarrow r + \gamma\, V(s') - V(s)$
    ✦ $e(s) \leftarrow e(s) + 1$ *(accumulating trace)* ✦ (Eligibility traces — TD(λ))
    ✦ **for all** $\bar{s} \in S$ **do** $V(\bar{s}) \leftarrow V(\bar{s}) + \alpha\, \delta\, e(\bar{s})$; $e(\bar{s}) \leftarrow \gamma\lambda\, e(\bar{s})$ **end for** ✦ (Eligibility traces — TD(λ))
    ✦ $\alpha \leftarrow \alpha_0/(1 + \text{visits}(s)/c)$ *(per-state Robbins–Monro decay)* ✦ (Robbins–Monro step-size schedule)
    ✦ *(Off-policy + function approx: use Gradient-TD / TDC update to guarantee convergence)* ✦ (Gradient-TD (GTD/TDC))
    $s \leftarrow s'$
  until s is terminal
end for
Output: $V \approx v_\pi$

✦✦✦

1.5.6 n-Step TD

Model-free; bridges TD(0) and Monte Carlo — uses ***n-step rollouts*** *(full episodes only if* $n = \infty$*).*

- $n = 1$: equivalent to TD(0) (bootstrap after 1 step)
- $n = \infty$: equivalent to Monte Carlo (full episode return)

Notation: - n — lookahead depth (number of real steps before bootstrapping) - τ — index of the state currently being updated; lags n steps behind the current time t - T — terminal time step (recorded when episode ends) - $G_{t:t+n}$ — n-step return starting at step t: $r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$

Introduction: Generalisation of TD(0) that bootstraps after n real steps rather than 1. *Idea:* update toward the n-step return $G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$, which interpolates between TD(0) ($n = 1$, low variance / high bias) and Monte Carlo ($n = \infty$, no bootstrap, high variance / no bias). *Why it helps:* in practice a moderate n trades bias against variance better than either extreme.

Algorithm:

Input: policy π, step size α, discount γ, lookahead n, number of episodes M

1. Initialize $V(s)$ arbitrarily for all $s \in S$; $\quad V(\text{terminal}) = 0$
2. **For** episode $= 1, 2, \ldots, M$ *(outer loop — episodes)*:
   - Initialize s_0; $\quad T \leftarrow \infty$
   - **For** $t = 0, 1, 2, \ldots$ *(inner loop — time steps)*:
     - If $t < T$:
       - Take action $\pi(s_t)$, observe $r_{t+1},\ s_{t+1}$
       - If s_{t+1} is terminal: $T \leftarrow t + 1$
     - Set $\tau \leftarrow t - n + 1$ *(time step being updated)*
     - If $\tau \geq 0$:
       - **Compute n-step return:**

$$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,\, T)} \gamma^{i-\tau-1}\, r_i$$

       - If $\tau + n < T$: $G \leftarrow G + \gamma^n V(s_{\tau+n})$ *(bootstrap)*
       - **Update:** $\quad V(s_\tau) \leftarrow V(s_\tau) + \alpha\,[G - V(s_\tau)]$
   - Until $\tau = T - 1$
3. Return V

✦ ✦ ✦

Notes:

- The n-step return: $G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$
- Updates are *delayed* by n steps (must wait n steps before updating s_t).
- If the episode ends before n steps, the return is a full MC return (no bootstrap).
- Larger $n \Rightarrow$ lower bias, higher variance. Smaller $n \Rightarrow$ higher bias, lower variance.

Additional Known Engineering Tricks

- **λ-return (forward view)** *(Sutton & Barto, §12.1)* — replace the single fixed-n return by a geometric average $G_t^\lambda = (1-\lambda)\sum_{k\geq 1}\lambda^{k-1}G_{t:t+k}$. *Why it helps:* a soft mixture over all n-step returns gives a smoother bias–variance trade-off than picking one n; tuning $\lambda \in [0,1]$ is much less sensitive than tuning a discrete n. *Interaction:* the *backward-view* equivalent uses eligibility traces (next bullet), giving an $O(|S|)$-per-step online update.
- **Backward-view equivalence via eligibility traces** — instead of waiting n steps and looking forward, maintain $e(s)$ traces and update *all* states at every step. *Why it helps:* removes the explicit n-step delay and the buffer of past states/rewards — online, $O(1)$-amortised update per step, identical fixed points to the forward view.
- **Tree-backup algorithm** *(Precup, 2000)* — for off-policy n-step without importance sampling, replace the bootstrap leaf with the expected value under the *target* policy and the action-probability of the on-policy action with the *behaviour* policy. *Why it helps:* enables off-policy n-step learning with bounded variance; classical n-step IS has variance that explodes with n.
- **Per-decision importance sampling** *(Precup et al., 2000)* — when off-policy, apply the IS ratio only on steps where the chosen action probability differs, instead of multiplying ratios over all n steps. *Why it helps:* keeps the estimator unbiased while drastically reducing variance vs. ordinary IS.
- **Retrace(λ)** *(Munos et al., 2016)* — clip the per-step IS ratio at 1 and combine with λ-returns: $c_k = \lambda \min(1, \pi/\mu)$. *Why it helps:* gives a *safe and efficient* off-policy n-step estimator — uniformly low variance, convergent for any μ, and used in modern off-policy actor-critic (ACER, IMPALA).
- **Circular ring-buffer for the last n transitions** — store (s, r) in two fixed-size n-element arrays with a rolling index. *Why it helps:* removes per-step allocation churn; the inner $G \leftarrow \sum \gamma^i r$ becomes a single dot-product. *Interaction:* the pseudocode's references to s_τ and r_i are abstract — in practice you cannot keep the full history, so a length-n rolling buffer is the standard implementation.

Algorithm 2.3 n-step TD for estimating v_π — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §7.1]*

Here is the pseudocode for the n-step TD core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: policy π, step size $\alpha \in (0, 1]$, discount γ, lookahead n, episodes M
Initialize $V(s)$ arbitrarily; $V(\text{terminal}) \leftarrow 0$
for episode $= 1, 2, \ldots, M$ **do**
  Initialize and store s_0; $T \leftarrow \infty$
  for $t = 0, 1, 2, \ldots$ **do**
    if $t < T$ **then**
      Take action $a_t \sim \pi(\cdot \mid s_t)$; observe r_{t+1}, s_{t+1}
      if s_{t+1} is terminal **then** $T \leftarrow t + 1$
    end if
    $\tau \leftarrow t - n + 1$ *(index of state being updated)*

if $\tau \geq 0$ **then**

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,\, T)} \gamma^{\,i-\tau-1}\, r_i$

if $\tau + n < T$ **then** $G \leftarrow G + \gamma^n\, V(s_{\tau+n})$

$V(s_\tau) \leftarrow V(s_\tau) + \alpha\,[\,G - V(s_\tau)\,]$

end if

until $\tau = T - 1$

end for

Algorithm 2.3 n-step TD for estimating v_π — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §7.1]*

Here is the pseudocode for the n-step TD algorithm with also employing all of the additions and engineering tricks mentioned above:

Input: policy π, step size α, discount γ, ✦ trace decay λ ✦ (λ-return (forward view)), lookahead n, episodes M

Initialise $V(s)$ arbitrarily; $V(\text{terminal}) \leftarrow 0$

for episode $= 1, 2, \ldots, M$ **do**

Initialise and store s_0; $T \leftarrow \infty$

✦ Allocate circular ring-buffer buf$[0 \ldots n{-}1]$ for (s, r) pairs ✦ (Circular ring-buffer for the last n transitions)

✦ $e(s) \leftarrow 0$ for all s *(eligibility traces — backward-view equivalent)* ✦ (Backward-view equivalence via eligibility traces)

for $t = 0, 1, 2, \ldots$ **do**

if $t < T$ **then**

Take action $a_t \sim \pi(\cdot \mid s_t)$; observe r_{t+1}, s_{t+1}

✦ Store (s_t, r_{t+1}) in buf$[t \bmod n]$ ✦ (Circular ring-buffer for the last n transitions)

if s_{t+1} is terminal **then** $T \leftarrow t + 1$

end if

$\tau \leftarrow t - n + 1$

if $\tau \geq 0$ **then**

✦ $G \leftarrow G_\tau^\lambda = (1-\lambda)\sum_{k\geq 1} \lambda^{k-1} G_{\tau:\tau+k}$ *(λ-return mixing all n-step targets)* ✦ (λ-return (forward view))

✦ *(Off-policy: apply per-decision IS ratios $\rho_i = \pi(a_i \mid s_i)/\mu(a_i \mid s_i)$ to each step)* ✦ (Per-decision importance sampling)

✦ *(Off-policy alternative: use Retrace(λ) with $c_k = \lambda \min(1, \pi/\mu)$ for safe correction)* ✦ (Retrace(λ))

✦ *(Off-policy alternative: tree-backup — no IS ratios, replace leaf with $\mathbb{E}_\pi[Q]$)* ✦ (Tree-backup algorithm)

$V(s_\tau) \leftarrow V(s_\tau) + \alpha\,[\,G - V(s_\tau)\,]$

✦ *(Backward-view equivalent: update all states via traces $e(\bar{s}) \leftarrow \gamma\lambda\, e(\bar{s})$; $V(\bar{s})$ += $\alpha\delta\, e(\bar{s})$)* ✦ (Backward-view equivalence via eligibility traces)

end if

until $\tau = T - 1$

end for

✦✦✦

1.5.7 SARSA (State-Action-Reward-State-Action)

*Model-free, on-policy, TD(0)-based control (learns Q directly); bootstraps after **each step** (no need for full episodes). Named after the quintuple:* (s, a, r, s', a').

Notation: - α — step size (learning rate) - s' — next state after taking a - a' — next action, chosen by the current policy from s' *before* the update is applied (hence the name SARSA)

Introduction: Model-free, ***on-policy*** TD control. Named after the quintuple (s, a, r, s', a') that drives each update. *Idea:* update $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s', a') - Q(s,a)\big]$ using the action a' that the **current** (ε-greedy) policy actually chooses from s'. *Why it helps:* on-policy estimates account for exploration in the target, so SARSA learns the value of the policy it actually follows — preferring safer paths than Q-learning when exploration noise is non-trivial (the canonical cliff-walking example).

Algorithm:

Input: step size $\alpha \in (0, 1]$, discount γ, exploration rate ε, episodes n

1. Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A$; $\quad Q(\text{terminal}, \cdot) = 0$
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - Initialize s; choose a from s using ε-greedy w.r.t. Q
   - **Repeat** for each step *(inner loop — time steps)*:
     - Take action a, observe $r,\ s'$
     - Choose a' from s' using ε-greedy w.r.t. Q *(choose next action now)*
     - $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma\, Q(s', a') - Q(s,a)]$ *(uses a' from the **same** policy — on-policy)*
     - $s \leftarrow s'; \quad a \leftarrow a'$
   - Until s is terminal
3. Return Q; derive $\pi(s) = \arg\max_a Q(s,a)$

✦ ✦ ✦

Notes:

- **On-policy:** the action a' used in the TD target $Q(s', a')$ is chosen by the **same** ε-greedy policy that the agent is following. The agent learns about the policy it is actually using (including its exploration).
- Because it accounts for exploration, SARSA tends to learn **safer / more conservative** policies than Q-learning in stochastic environments.
- The update uses $Q(s', a')$ where a' is already chosen — hence the name s, a, r, s', a'.

Additional Known Engineering Tricks

- **Expected SARSA** *(van Seijen et al., 2009)* — replace the sampled $Q(s', a')$ in the TD target with the expectation under π: $\quad y = r + \gamma \sum_{a'} \pi(a' \mid s')\, Q(s', a')$. *Why it helps:* removes the sampling variance of a' at no bias cost — a strict improvement over standard SARSA for the same number of samples. *Interaction:* recovers Q-learning when π is greedy and recovers SARSA when π is the sampling policy.
- **SARSA(λ)** — eligibility traces over (s, a) pairs, decayed by $\gamma\lambda$; on each step update *every* (s, a) visited recently by $\alpha\delta\, e(s, a)$. *Why it helps:* much faster credit assignment over long

episodes than one-step SARSA. *Interaction:* watch out — combining with ε-greedy makes the trace technically off-policy on exploratory actions; the common Watkins-style fix is to *cut* the trace whenever a non-greedy action is taken.

- **Boltzmann (softmax) exploration** — sample $a \sim \text{softmax}(Q(s,\cdot)/\tau)$ instead of ε-greedy. *Why it helps:* allocates exploration in proportion to perceived value rather than uniformly over non-greedy actions; usually safer for SARSA's *on-policy* value, which directly reflects the cost of suboptimal exploration. *Interaction:* mutually exclusive with ε-greedy; temperature τ is annealed similarly to ε.
- **n-step SARSA** — use the n-step on-policy return $G_{t:t+n} = r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n Q(s_{t+n}, a_{t+n})$ as the target. *Why it helps:* the same bias-variance trade-off as n-step TD prediction, but for control; smooths SARSA's reaction time without forcing full Monte-Carlo episodes.
- **Double SARSA / Double Expected SARSA** — maintain Q^A, Q^B; on each step randomly pick one to update using the other's value at (s', a'). *Why it helps:* decouples action selection from action evaluation, breaking the maximisation-bias problem when Q contains noise; same idea as Double Q-learning but for the on-policy target.
- **Optimistic initialisation** — set $Q(s,a)$ high so unvisited pairs look attractive. *Why it helps:* drives systematic exploration without explicit ε; especially effective in SARSA because the on-policy target dampens the optimism gracefully as actions are tried.

Algorithm 2.4 SARSA (on-policy TD control) — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §6.4]*

Here is the pseudocode for the SARSA core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: step size $\alpha \in (0, 1]$, discount γ, exploration rate ε, episodes n
Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A;\ Q(\text{terminal}, \cdot) = 0$
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  Choose a from s using ε-greedy w.r.t. Q
  repeat (for each step of episode)
    Take action a; observe $r,\ s'$
    Choose a' from s' using ε-greedy w.r.t. Q
    $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma\, Q(s',a') - Q(s,a)\,]$
    $s \leftarrow s'; \quad a \leftarrow a'$
  until s is terminal
end for
Output: $Q \approx q_*$

Algorithm 2.4 SARSA (on-policy TD control) — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §6.4]*

Here is the pseudocode for the SARSA algorithm with also employing all of the additions and

engineering tricks mentioned above:
Input: step size α, discount γ, ✦ temperature τ ✦ (Boltzmann (softmax) exploration), ✦ trace decay λ ✦ (SARSA(λ)), ✦ lookahead n ✦ (n-step SARSA), episodes M
✦ Initialise $Q^A(s,a), Q^B(s,a) \leftarrow R_{\max}/(1-\gamma)$ for all s, a *(optimistic; two tables for Double SARSA)* ✦ (Optimistic initialisation / Double SARSA / Double Expected SARSA)
$Q(\text{terminal}, \cdot) = 0$
for episode $= 1, 2, \ldots, M$ **do**
  Initialise s
  ✦ $e(s,a) \leftarrow 0$ for all s, a *(eligibility traces)* ✦ (SARSA(λ))
  ✦ Choose $a \sim \text{softmax}(Q(s,\cdot)/\tau)$ ✦ (Boltzmann (softmax) exploration)
  repeat (for each step of episode)
    Take action a; observe r, s'
    ✦ Choose $a' \sim \text{softmax}(Q(s',\cdot)/\tau)$ ✦ (Boltzmann (softmax) exploration)
    ✦ *(n-step variant: accumulate* $G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n Q(s_{t+n}, a_{t+n})$ *before updating)* ✦ (n-step SARSA)
    ✦ $y \leftarrow r + \gamma \sum_{a''} \pi(a'' \mid s') Q(s', a'')$ *(Expected SARSA target — replaces sampled* $Q(s',a')$*)* ✦ (Expected SARSA)
    $\delta \leftarrow y - Q(s,a)$
    ✦ $e(s,a) \leftarrow e(s,a) + 1$ ✦ (SARSA(λ))
    ✦ **for all** $(\bar{s}, \bar{a})$ **do** $Q(\bar{s},\bar{a}) \leftarrow Q(\bar{s},\bar{a}) + \alpha\,\delta\,e(\bar{s},\bar{a})$; $e(\bar{s},\bar{a}) \leftarrow \gamma\lambda\,e(\bar{s},\bar{a})$ **end for** ✦ (SARSA(λ))
    ✦ *(Double variant: with 50 % prob. update* Q^A *using* Q^B*'s value at* (s',a')*, else vice versa)* ✦ (Double SARSA / Double Expected SARSA)
    $s \leftarrow s'$; $\quad a \leftarrow a'$
  until s is terminal
end for
Output: $Q \approx q_*$

✦✦✦

1.5.8 Q-Learning

*Model-free, off-policy, TD(0)-based control (learns Q directly); bootstraps after **each step** (no need for full episodes).*

Notation: - α — step size (learning rate) - s' — next state after taking a - a' — dummy variable ranging over all actions at s'; only used inside $\max_{a'}$ and never actually taken

Introduction: Model-free, ***off-policy*** TD control — learns the optimal action-value function Q^* regardless of the behaviour policy. *Idea:* update $Q(s,a) \leftarrow Q(s,a)+\alpha[r+\gamma \max_{a'} Q(s',a')-Q(s,a)]$ using the ***greedy*** action at s', even though the behaviour policy may have explored elsewhere. *Why it helps:* decouples behaviour and target — the agent can explore widely (or follow any policy that covers $S \times A$) while the learnt Q still converges to Q^*.

Algorithm:

Input: step size $\alpha \in (0,1]$, discount γ, exploration rate ε, episodes n

1. Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A$; $\quad Q(\text{terminal},\cdot) = 0$
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - Choose a from s using ε-greedy w.r.t. Q *(behavior policy — explores)*
     - Take action a, observe $r,\ s'$
     - $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ *(uses **greedy max**, not the action actually taken)*
     - $s \leftarrow s'$
   - Until s is terminal
3. Return Q; derive $\pi(s) = \arg\max_a Q(s,a)$

✦ ✦ ✦

Notes:

- **Off-policy:** the agent follows an ε-greedy behavior policy (to explore), but the TD target uses $\max_{a'} Q(s',a')$ — the **greedy** action — regardless of what the agent actually takes next. Q-learning directly learns Q^* even while exploring.
- **Key difference from SARSA:**

	TD update target
SARSA	$r + \gamma\, Q(s',a')$ where $a' \sim \varepsilon$-greedy
Q-learning	$r + \gamma \max_{a'} Q(s',a')$ (greedy max)

- Q-learning can be more sample-efficient but may learn riskier policies than SARSA in stochastic environments (it ignores the cost of exploration).

Additional Known Engineering Tricks

- **Double Q-learning** *(van Hasselt, 2010)* — maintain two value tables Q^A, Q^B; on each step, with 50% probability update $Q^A(s,a) \leftarrow Q^A(s,a) + \alpha[r + \gamma\, Q^B(s', \arg\max_{a'} Q^A(s',a')) - Q^A(s,a)]$ (and symmetrically). *Why it helps:* the $\max_{a'}$ in standard Q-learning systematically

overestimates the action value because of noise; using a *different* table to evaluate the action selected by the first cancels this bias. *Interaction:* foundational idea behind Double DQN in deep RL.

- **Watkins's Q(λ) / Peng's Q(λ)** *(Watkins, 1989; Peng & Williams, 1996)* — eligibility traces for off-policy Q-learning. Watkins's variant *cuts* the trace to zero whenever a non-greedy action is taken (preserves convergence); Peng's leaves the trace alive (faster but biased). *Why it helps:* propagates credit over multi-step trajectories without waiting; Watkins is the safe default. *Interaction:* the trace-cut is necessary precisely *because* Q-learning is off-policy — see also the *retrace* fix in the n-step TD section.
- **Tile coding / state aggregation** *(CMAC, Albus 1975)* — partition continuous s into overlapping tilings of bins and use the indicator vector as features; Q becomes a weighted sum over active tiles. *Why it helps:* a simple, theoretically-grounded linear function approximation that scales tabular Q-learning to medium-dimensional continuous state spaces with provable convergence (for on-policy update; care needed off-policy).
- **Count-based / UCB-style exploration** — replace ε-greedy with $a \leftarrow \arg\max_a [Q(s,a) + c\sqrt{\log t / N(s,a)}]$, where $N(s,a)$ is the visit count. *Why it helps:* directs exploration to truly under-sampled actions instead of random ones, dramatically improving sample efficiency in sparse-reward MDPs.
- **Reward shaping** *(Ng, Harada & Russell, 1999)* — add a *potential-based* shaping reward $F(s,a,s') = \gamma\Phi(s') - \Phi(s)$ to the environment reward. *Why it helps:* injects domain knowledge to densify the reward signal without changing the optimal policy (the potential telescopes out). Critically, only *potential-based* shaping is policy-invariant — arbitrary shaping can change the optimum.
- **Optimistic Q initialisation** — initialise $Q(s,a) = R_{\max}/(1-\gamma)$. *Why it helps:* drives early exploration to under-visited actions even with greedy selection; less wasteful than ε-greedy on tasks with a clear gradient of reward.

Algorithm 2.5 Q-learning (off-policy TD control) — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §6.5; Watkins, 1989]*

Here is the pseudocode for the Q-learning core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: step size $\alpha \in (0,1]$, discount γ, exploration rate ε, episodes n
Initialize $Q(s,a)$ arbitrarily for all $s \in S,\ a \in A;\ Q(\text{terminal},\cdot) = 0$
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  repeat (for each step of episode)
    Choose a from s using ε-greedy w.r.t. Q *(behavior policy)*
    Take action a; observe $r,\ s'$
    $Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma\ \max_{a'} Q(s',a') - Q(s,a)\right]$
    $s \leftarrow s'$
  until s is terminal
end for
Output: $Q \approx q_*$

Algorithm 2.5 Q-learning (off-policy TD control) — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §6.5; Watkins, 1989]*

Here is the pseudocode for the Q-learning algorithm with also employing all of the additions and engineering tricks mentioned above:

Input: step size α, discount γ, ✦ trace decay λ ✦ (Watkins's Q(λ) / Peng's Q(λ)), ✦ UCB constant c ✦ (Count-based / UCB-style exploration), ✦ potential function $\Phi(s)$ ✦ (Reward shaping), episodes n

✦ Initialise $Q^A(s,a), Q^B(s,a) \leftarrow R_{\max}/(1-\gamma)$ *(optimistic; two tables for Double Q)* ✦ (Optimistic Q initialisation / Double Q-learning)

$Q(\text{terminal}, \cdot) = 0$

✦ $N(s,a) \leftarrow 0$ for all s, a *(visit counts for UCB)* ✦ (Count-based / UCB-style exploration)

✦ *(Alternative: replace tabular Q with tile coding — $Q(s,a) = \mathbf{w}^\top \phi(s,a)$ over overlapping tilings)* ✦ (Tile coding / state aggregation)

for episode $= 1, 2, \ldots, n$ **do**

Initialise s

✦ $e(s,a) \leftarrow 0$ for all s, a *(eligibility traces)* ✦ (Watkins's Q(λ) / Peng's Q(λ))

repeat (for each step of episode)

✦ $a \leftarrow \arg\max_a \left[Q(s,a) + c\sqrt{\log t / N(s,a)}\right]$ *(UCB action selection)* ✦ (Count-based / UCB-style exploration)

Take action a; observe r, s'

✦ $N(s,a) \leftarrow N(s,a) + 1$ ✦ (Count-based / UCB-style exploration)

✦ $r' \leftarrow r + \gamma\,\Phi(s') - \Phi(s)$ *(potential-based reward shaping)* ✦ (Reward shaping)

✦ With prob. 0.5: $a^* \leftarrow \arg\max_{a'} Q^A(s',a')$; $Q^A(s,a) \leftarrow Q^A(s,a) + \alpha\left[r' + \gamma\, Q^B(s',a^*) - Q^A(s,a)\right]$; else swap A/B ✦ (Double Q-learning)

✦ $e(s,a) \leftarrow e(s,a) + 1$; **if** a was greedy: decay $e \leftarrow \gamma\lambda\, e$; **else** cut $e \leftarrow 0$ *(Watkins)* ✦ (Watkins's Q(λ) / Peng's Q(λ))

✦ **for all** $(\bar{s}, \bar{a})$ **do** $Q(\bar{s},\bar{a})$ += $\alpha\,\delta\, e(\bar{s},\bar{a})$ **end for** ✦ (Watkins's Q(λ) / Peng's Q(λ))

$s \leftarrow s'$

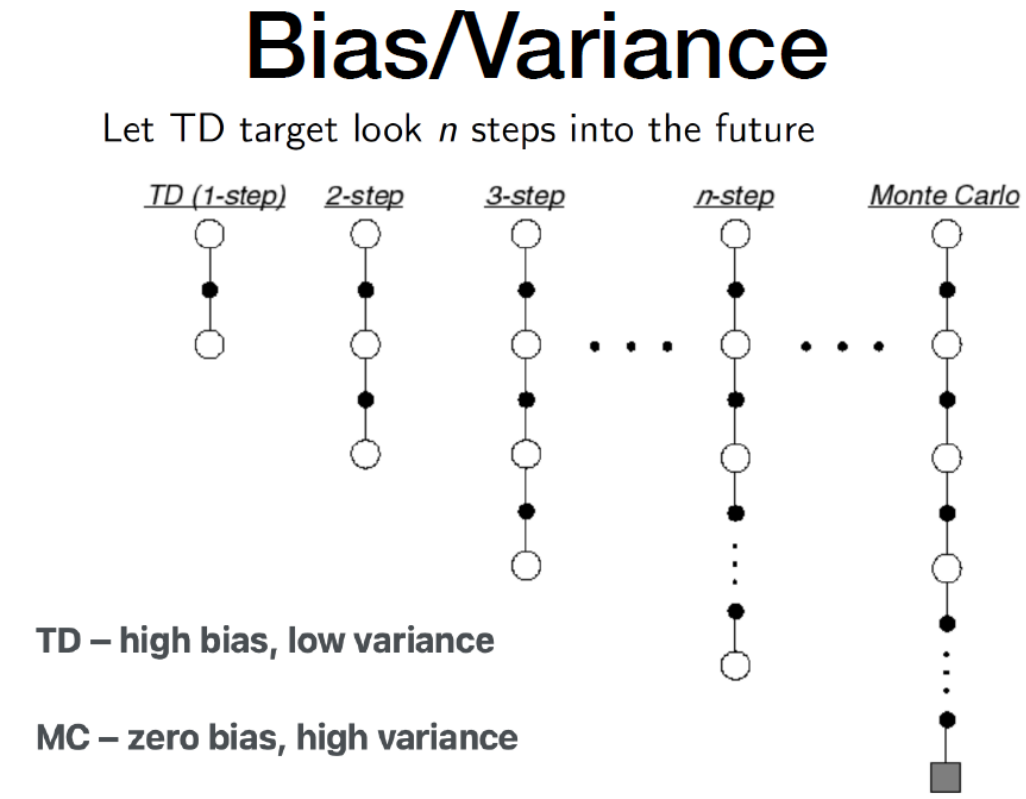
until s is terminal

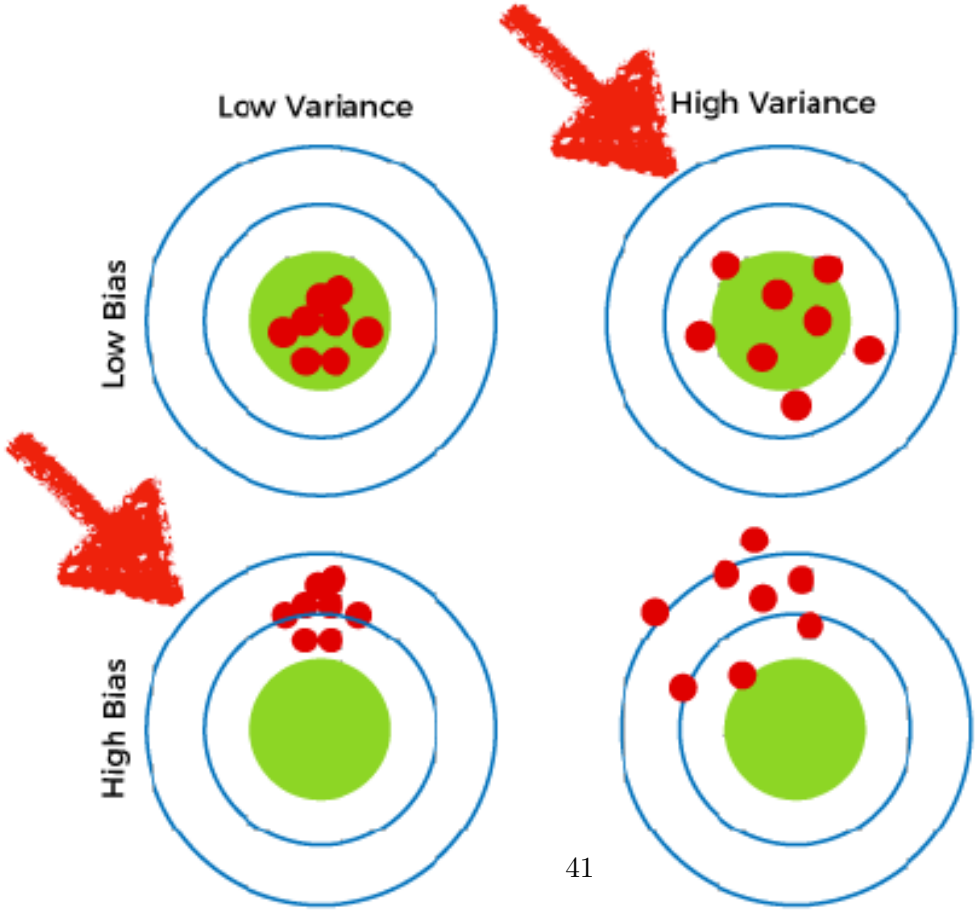
end for

Output: $Q \approx q_*$

✦✦✦

1.5.9 Summary facts





✦ ✦ ✦

1.6 Deep RL — Value-Based

✦ ✦ ✦

1.6.1 DQN Naive (Deep Q-Network, no tricks)

Notation: - θ — neural network weights (learnable parameters; *not* a convergence threshold here) - $Q(s, a; \theta)$ — Q-value approximated by a neural net with weights θ - $\mathcal{L}(\theta)$ — loss function (squared Bellman error) - ∇_θ — gradient with respect to θ

Introduction: Q-learning with a neural network $Q(s, a; \theta)$ in place of the tabular Q. *Idea:* minimise the squared one-step Bellman error $\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta)\right)^2$ via stochastic gradient descent. *Why it helps:* scales Q-learning to large/continuous state spaces (e.g. pixel inputs) where a Q-table is infeasible — but this naive form is unstable because the target and the prediction share the same rapidly changing weights θ.

Algorithm:

Input: step size α, discount γ, exploration rate ε, episodes n

1. Initialize neural network $Q(s, a; \theta)$ with random weights θ
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - Choose a using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
     - Take action a, observe $r,\ s'$
     - **Compute target** *(uses the **same** network θ)*:

$$y \leftarrow r + \gamma \max_{a'} Q(s', a';\, \theta)$$

     - **Gradient update:**

$$\mathcal{L}(\theta) = \left(y - Q(s, a; \theta)\right)^2, \qquad \theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

     - $s \leftarrow s'$
   - Until s is terminal
3. Return $Q(\cdot, \cdot; \theta)$; derive $\pi(s) = \arg\max_a Q(s, a; \theta)$

✦ ✦ ✦

Problems with Naive DQN:

- **Instability:** the target y depends on the same θ being updated — every gradient step shifts both the prediction and the target ⇒ *moving target problem.*
- **Correlated samples:** consecutive transitions (s, a, r, s') are highly correlated; SGD assumes i.i.d. samples ⇒ this assumption is violated.
- These two issues cause divergence or oscillation in practice.

⇒ Fixed by **Target Network** and **Experience Replay** (see below).

Additional Known Engineering Tricks

- **Reward clipping to** $[-1, +1]$ *(Mnih et al., 2013)* — clip the per-step reward into a fixed range before the TD target is computed. *Why it helps:* puts all Atari games (or other tasks with wildly different reward magnitudes) on a single learning-rate scale; prevents a single jackpot reward from blowing up the gradient. *Interaction:* changes the *value of* γ *effectively* (since rewards are bounded), so combine with the standard $\gamma = 0.99$.
- **Frame skipping / action repeat** — repeat the same action for $k = 4$ environment steps and only call the network on every k-th frame. *Why it helps:* cuts the inference cost by k and increases the effective discount horizon at no quality cost in environments with high temporal correlation between successive frames.
- **Frame stacking** — concatenate the last $k = 4$ pre-processed frames into a single k-channel observation. *Why it helps:* injects a velocity signal into an otherwise-Markovian-in-pixels state; the Q-network can then infer motion without recurrence. *Interaction:* combines tightly with frame skipping — the stack contains k skipped frames, giving a k^2-frame temporal window.
- **Atari pre-processing pipeline** — grayscale, resize to 84×84, and take the *pixel-wise max* over two adjacent raw frames to handle Atari's odd–even sprite flickering. *Why it helps:* drops input dimensionality by $\sim 30\times$ and removes sprite-flicker noise that would otherwise look like a non-Markovian observation.
- **Huber loss (smooth L1)** instead of MSE — quadratic for $|y - Q| \leq 1$, linear beyond. *Why it helps:* equivalent to clipping the TD error gradient at ± 1 — bounds the magnitude of a single bad target so a noisy y cannot blow up the parameters of the value head.
- **RMSProp with low learning rate** *(Mnih et al., 2013 —* $\alpha = 2.5 \times 10^{-4}$*, momentum* 0.95*,* $\epsilon = 10^{-2}$*)* — explicit per-parameter step normalisation. *Why it helps:* compensates for highly-variable gradient scales across the conv stack; the specific small-α + large-ϵ combo is what made the *naive* (no target net, no replay) DQN converge at all on simple Atari games. *Interaction:* once target net + replay are added (next sections), Adam with $\alpha \sim 10^{-4}$ works fine and supersedes this choice.

Algorithm 3.1 DQN (naive — no target net, no replay) — Raw Pseudocode (without additional tricks) *[Mnih et al., 2013]*

Here is the pseudocode for the DQN (naive) core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: step size α, discount γ, exploration rate ε, episodes n
Initialize Q-network $Q(s, a; \theta)$ with random weights θ
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  repeat (for each step of episode)
    Choose a from s using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
    Take action a; observe r, s'

$y \leftarrow r + \gamma \max_{a'} Q(s', a'; \theta)$ *(bootstrap from same network)*
$\mathcal{L}(\theta) \leftarrow \big(y - Q(s, a; \theta) \big)^2$
$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$
$s \leftarrow s'$
until s is terminal
end for

Algorithm 3.1 DQN (naive — no target net, no replay) — Pseudocode with Additions (Engineering Tricks) *[Mnih et al., 2013]*

Here is the pseudocode for the DQN (naive) algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: ✦ RMSProp with $\alpha = 2.5 \times 10^{-4}$, momentum 0.95, $\epsilon_{\text{opt}} = 10^{-2}$ ✦ (RMSProp with low learning rate), discount γ, exploration ε, episodes n, ✦ action repeat $k = 4$ ✦ (Frame skipping / action repeat)
Initialise Q-network $Q(s, a; \theta)$ with random weights θ
for episode $= 1, 2, \ldots, n$ **do**
Initialise s
✦ Pre-process s: grayscale, resize to 84×84, pixel-max over last 2 raw frames ✦ (Atari pre-processing pipeline)
✦ Stack last $k = 4$ pre-processed frames as channels ✦ (Frame stacking)
repeat (for each step of episode)
Choose a from s using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
✦ Repeat action a for $k = 4$ environment steps; accumulate reward ✦ (Frame skipping / action repeat)
Take action a; observe r, s'
✦ $r \leftarrow \text{clip}(r, -1, +1)$ ✦ (Reward clipping to $[-1, +1]$)
✦ Pre-process and frame-stack s' ✦ (Atari pre-processing pipeline / Frame stacking)
$y \leftarrow r + \gamma \max_{a'} Q(s', a'; \theta)$
✦ $\mathcal{L}(\theta) \leftarrow \text{Huber}(y - Q(s, a; \theta))$ *(smooth L1 — linear for $|e| > 1$, quadratic otherwise)* ✦ (Huber loss (smooth L1))
✦ $\theta \leftarrow \theta - \text{RMSProp}(\nabla_\theta \mathcal{L})$ ✦ (RMSProp with low learning rate)
$s \leftarrow s'$
until s is terminal
end for

✦✦✦

1.6.2 DQN + Target Network (DQN+TN)

Notation: - θ^- — frozen target network weights (a periodic hard copy of θ; never updated by gradients) - $\hat{Q}(s, a; \theta^-)$ — target network (separate from the online network Q) - C — target update frequency (gradient steps between hard copies $\theta^- \leftarrow \theta$) - t — global step counter

Introduction: DQN with a ***separate frozen target network*** $\hat{Q}(s, a; \theta^-)$ used to compute the Bellman target; θ^- is a periodic hard copy of the online weights θ. *Idea:* compute the target with θ^- instead of θ so the target stays still while the online network chases it, then sync $\theta^- \leftarrow \theta$ every C steps. *Why it helps:* fixes the moving-target problem that destabilises naive DQN — divergence/oscillation are replaced by a steady (if slightly stale) regression target.

Algorithm:

Input: α, γ, ε, episodes n, target update frequency C

1. Initialize **online network** $Q(s, a; \theta)$ with random weights θ

   Initialize **target network** $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$ *(copy of online network)*

   Initialize step counter $t \leftarrow 0$

2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:

   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - Choose a using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
     - Take action a, observe r, s'
     - **Compute target using frozen target network** *(θ^- is NOT updated here)*:

$$y \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a';\, \theta^-)$$

     - **Update online network:**

$$\mathcal{L}(\theta) = \big(y - Q(s, a; \theta)\big)^2, \qquad \theta \leftarrow \theta - \alpha\, \nabla_\theta\, \mathcal{L}(\theta)$$

     - $t \leftarrow t + 1$; if $t \bmod C = 0$: $\theta^- \leftarrow \theta$ *(hard copy every C steps)*
     - $s \leftarrow s'$
   - Until s is terminal

3. Return $Q(\cdot, \cdot; \theta)$; derive $\pi(s) = \arg\max_a Q(s, a; \theta)$

✦ ✦ ✦

Notes:

- θ^- is **frozen** between updates ⇒ target y does not shift with every gradient step ⇒ stabilizes learning.
- Alternative: **soft update** $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ with small τ (e.g., $\tau = 0.005$) every step.
- This alone still suffers from correlated samples ⇒ combine with Experience Replay.

Additional Known Engineering Tricks

- **Polyak (soft) target update** — $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ every step with small τ (e.g., 0.005). *Why it helps:* smoothly tracks the online network with controlled lag instead of taking a

sudden jump every C steps; the effective time constant is $1/\tau \approx 200$. *Interaction:* mathematically interpolates between hard-sync ($\tau = 1$ every step) and frozen target ($\tau = 0$); pick one of hard/soft, not both. Mandatory in DDPG/SAC where the actor's gradient flows through the target critic and sudden jumps would destabilise the actor.

- **Target network *only* for $\max_{a'} Q$, not for action selection at s** — keep $a \sim \varepsilon$-greedy$(Q(s, \cdot; \theta))$ from the *online* net. *Why it helps:* using θ^- for behaviour too would make exploration stale by C steps; using θ for behaviour and θ^- only inside the regression target gives both fresh exploration and a stable regression problem.
- **Asymmetric C scheduling** — start with small C ($\approx$100) during the noisy early phase and increase to large C later. *Why it helps:* early on, the network is changing fast and a tight C tracks reality; later, learning is finer and benefits from longer-frozen targets to reduce target jitter.
- **Decoupling the max** *(precursor to Double DQN)* — when the target network is in place, you have two networks lying around; use the online net to *select* the maximising action and the target net to *evaluate* it: $y = r + \gamma\, \hat{Q}(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$. *Why it helps:* same maximisation-bias fix as Double Q-learning, free from the existing two-network setup; ships as **Double DQN** in the full algorithm.
- **Target net for $V(s')$ in n-step bootstraps** — when extending to n-step DQN, evaluate the bootstrap leaf $Q(s_{t+n}, \cdot)$ with θ^- but compute the running sum of rewards with no network involvement. *Why it helps:* keeps the only network call in the target stable; the reward sum is data-only.

Algorithm 3.2 DQN with target network — Raw Pseudocode (without additional tricks) *[Mnih et al., 2015]*

Here is the pseudocode for the DQN with target network core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: α, γ, ε, episodes n, target update frequency C
Initialize online network $Q(s, a; \theta)$
Initialize target network $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$
Step counter $t \leftarrow 0$
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  repeat (for each step of episode)
    Choose a from s using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
    Take action a; observe r, s'
    $y \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)$ *(target from frozen net)*
    $\mathcal{L}(\theta) \leftarrow \big(y - Q(s, a; \theta)\big)^2$
    $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$
    $s \leftarrow s'$; $\quad t \leftarrow t + 1$
    if $t \bmod C = 0$ **then** $\theta^- \leftarrow \theta$
  until s is terminal
end for

Algorithm 3.2 DQN with target network — Pseudocode with Additions (Engineering Tricks) *[Mnih et al., 2015]*

Here is the pseudocode for the DQN with target network algorithm with also employing all of the additions and engineering tricks mentioned above:

Input: α, γ, ε, episodes n, ✦ soft-update τ ✦ (Polyak (soft) target update), ✦ initial C_0, final C_{final} ✦ (Asymmetric C scheduling), ✦ bootstrap steps m ✦ (Target net for $V(s')$ in n-step bootstraps)

Initialise online network $Q(s, a; \theta)$

Initialise target network $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$

Step counter $t \leftarrow 0$

for episode $= 1, 2, \ldots, n$ **do**

  Initialise s

  repeat (for each step of episode)

    ✦ Choose a from s using ε-greedy w.r.t. $Q(s, \cdot; \theta)$ *(online net for behaviour, not target)* ✦ (Target network *only* for $\max_{a'} Q$, not for action selection at s)

    Take action a; observe r, s'

    ✦ $a^* \leftarrow \arg\max_{a'} Q(s', a'; \theta)$ *(online net selects)* ✦ (Decoupling the max)

    ✦ $y \leftarrow r + \gamma\, \hat{Q}(s', a^*; \theta^-)$ *(target net evaluates — Double DQN)* ✦ (Decoupling the max)

    ✦ *(For m-step bootstrap: $y \leftarrow \sum_{k=0}^{m-1} \gamma^k r_{t+k} + \gamma^m \hat{Q}(s_{t+m}, \cdot; \theta^-)$)* ✦ (Target net for $V(s')$ in n-step bootstraps)

    $\mathcal{L}(\theta) \leftarrow \big(\, y - Q(s, a; \theta)\,\big)^2$

    $\theta \leftarrow \theta - \alpha\, \nabla_\theta\, \mathcal{L}(\theta)$

    $s \leftarrow s'$; $t \leftarrow t + 1$

    ✦ $\theta^- \leftarrow \tau\, \theta + (1 - \tau)\, \theta^-$ *(Polyak soft update every step)* ✦ (Polyak (soft) target update)

    ✦ *(Alternative: hard sync every C steps, with C increasing over training)* ✦ (Asymmetric C scheduling)

  until s is terminal

end for

✦✦✦

1.6.3 DQN + Experience Replay (DQN+ER)

Notation: - $\mathcal{D}$ — replay buffer storing past transitions - M — buffer capacity (max transitions stored; oldest discarded when $|\mathcal{D}| > M$) - B — minibatch size (transitions sampled per gradient update) - done — boolean terminal flag: true iff s' is a terminal state

Same as DQN+TN above: - θ^- — frozen target network weights (a periodic hard copy of θ; never updated by gradients) - $\hat{Q}(s, a; \theta^-)$ — target network (separate from the online network Q) - C — target update frequency (gradient steps between hard copies $\theta^- \leftarrow \theta$) - t — global step counter

Introduction: The full DQN (Mnih et al., 2015). Adds an ***experience replay*** buffer on top of the target network: every transition $(s, a, r, s', \text{done})$ is stored in a FIFO buffer $\mathcal{D}$, and minibatches are sampled uniformly at random for each gradient update. *Idea:* train on shuffled past transitions instead of consecutive ones, and reuse each transition many times. *Why it helps:* decorrelates samples (consecutive trajectories badly violate the i.i.d. assumption SGD relies on) and dramatically improves sample efficiency by recycling data.

Algorithm:

Input: α, γ, ε, episodes n, buffer capacity M, minibatch size B, target update freq C

1. Initialize **online network** $Q(s, a; \theta)$; **target network** $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$

   Initialize replay buffer $\mathcal{D}$ with capacity M; step counter $t \leftarrow 0$

2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:

   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - Choose a using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
     - Take action a, observe $r,\ s',\ \text{done}$
     - Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$ *(FIFO: oldest removed if $|\mathcal{D}| > M$)*
     - Sample B transitions $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$ uniformly from $\mathcal{D}$
     - **Compute targets** for each j in minibatch:

$$y_j = \begin{cases} r_j & \text{if done}_j \\ r_j + \gamma \max_{a'} \hat{Q}(s'_j, a'; \theta^-) & \text{otherwise} \end{cases}$$

     - **Gradient update:**

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_j \big(y_j - Q(s_j, a_j; \theta)\big)^2, \qquad \theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

     - $t \leftarrow t + 1$; if $t \bmod C = 0$: $\theta^- \leftarrow \theta$
     - $s \leftarrow s'$
   - Until s is terminal (done = true)

3. Return $Q(\cdot, \cdot; \theta)$; derive $\pi(s) = \arg\max_a Q(s, a; \theta)$

✦ ✦ ✦

Notes:

- **Experience Replay** breaks temporal correlation: the minibatch contains transitions from different episodes and time steps ⇒ approximately i.i.d. samples for SGD.

- Each transition can be reused many times ⇒ better sample efficiency.
- $\mathcal{D}$ is a fixed-size FIFO queue (e.g., $M = 10^6$).
- ε is often annealed: start at 1.0 (pure exploration) → decay to 0.01 over training.
- This is the standard **full DQN** algorithm. In practice, both tricks are always used together.

Additional Known Engineering Tricks

- **Prioritized Experience Replay (PER)** *(Schaul et al., 2016)* — sample transition j with probability $\propto |\delta_j|^\alpha$ (the magnitude of its last TD error); correct the bias with importance-sampling weights $w_j = (1/(NP_j))^\beta$. *Why it helps:* high-error transitions carry more learning signal, so prioritising them speeds up training $\sim 2\times$ on Atari. *Interaction:* the IS weight is essential — without it the Q estimate becomes biased; sum-trees give $O(\log N)$ sampling.

- **Double DQN** *(van Hasselt, Guez & Silver, 2016)* — change the target to $y_j = r_j + \gamma\, \hat{Q}(s'_j, \arg\max_{a'} Q(s'_j, a'; \theta); \theta^-)$ — online net selects, target net evaluates. *Why it helps:* removes the systematic overestimation bias of $\max_{a'} \hat{Q}$ for free (no extra parameters). *Interaction:* needs a target network already, so combines trivially with the trick from the previous section.

- **Dueling network architecture** *(Wang et al., 2016)* — split the head into a state-value stream $V(s)$ and an advantage stream $A(s, a)$, combine as $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|A|}\sum_{a'} A(s, a'))$. *Why it helps:* lets the network learn the *value of being in a state* without committing to any particular action — important when actions don't affect the immediate value (e.g., navigating an empty corridor).

- **Noisy Networks for exploration** *(Fortunato et al., 2018)* — replace the linear layers in the Q-network with $y = (\mu_w + \sigma_w \odot \varepsilon_w)x + (\mu_b + \sigma_b \odot \varepsilon_b)$, with ε resampled each forward pass. *Why it helps:* learnable *parameter* noise replaces ε-greedy; exploration becomes state-conditional and self-tuning. *Interaction:* makes ε-greedy unnecessary and is required for Rainbow.

- **n-step bootstrap returns in the buffer** — store transitions as $(s_t, a_t, R_t^{(n)}, s_{t+n})$ with $R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1}$. *Why it helps:* faster reward propagation, same trade-off as n-step TD; typical $n = 3$ on Atari. *Interaction:* technically makes the target *slightly* off-policy when combined with ε-greedy, but in practice the small bias is dominated by faster convergence.

- **Distributional RL — C51 / QR-DQN / IQN** *(Bellemare et al., 2017; Dabney et al., 2018a,b)* — replace the scalar $Q(s, a)$ with a *distribution* over returns (51-bin categorical for C51; quantile regression for QR-DQN; implicit quantile for IQN). *Why it helps:* the network learns a richer signal (variance, skew, multimodality of returns), which empirically yields large gains; the policy is still $\arg\max_a \mathbb{E}[Z(s, a)]$.

- **Rainbow** *(Hessel et al., 2018)* — combines all six tricks above (PER + Double + Dueling + Noisy + n-step + Distributional) plus the basic target net + replay. *Why it helps:* every component fixes a different failure mode of naive DQN; ablation studies show they are largely complementary.

- **Replay warm-up / minimum buffer fill** — collect $\sim$50k random-policy transitions before any gradient update. *Why it helps:* ensures the very first minibatches are diverse enough not to overfit on a near-empty buffer; standard $|\mathcal{D}| = 10^6$ FIFO with min-size $\sim$5% of capacity.

Algorithm 3.3 Deep Q-learning with experience replay (full DQN) — Raw Pseudocode (without additional tricks) *[Mnih et al., 2015]*

Here is the pseudocode for the DQN with experience replay (full DQN) core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: α, γ, ε, episodes n, buffer capacity M, minibatch size B, target update freq C
Initialize online network $Q(s, a; \theta)$; target network $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$
Initialize replay buffer $\mathcal{D}$ with capacity M; step counter $t \leftarrow 0$
for episode $= 1, 2, \ldots, n$ **do**
  Initialize s
  repeat (for each step of episode)
    Choose a from s using ε-greedy w.r.t. $Q(s, \cdot; \theta)$
    Take action a; observe r, s', done
    Store transition $(s, a, r, s', \text{done})$ in $\mathcal{D}$
    Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$
    for each j **do**
      $y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)\,\max_{a'} \hat{Q}(s'_j, a'; \theta^-)$
    end for
    $\mathcal{L}(\theta) \leftarrow \dfrac{1}{B} \sum_{j=1}^{B} \big(y_j - Q(s_j, a_j; \theta) \big)^2$
    $\theta \leftarrow \theta - \alpha\, \nabla_\theta\, \mathcal{L}(\theta)$
    $s \leftarrow s'$; $t \leftarrow t + 1$
    if $t \bmod C = 0$ **then** $\theta^- \leftarrow \theta$
  until done $=$ **true**
end for

Algorithm 3.3 Deep Q-learning with experience replay (full DQN) — Pseudocode with Additions (Engineering Tricks) *[Mnih et al., 2015]*

Here is the pseudocode for the DQN with experience replay (full DQN) algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: α, γ, episodes n, buffer capacity M, minibatch B, target update freq C, ✦ PER exponent α_{per}, IS exponent β_{per} ✦ (Prioritized Experience Replay (PER)), ✦ bootstrap steps m ✦ (n-step bootstrap returns in the buffer), ✦ warm-up steps W ✦ (Replay warm-up / minimum buffer fill)
✦ Initialise Q-network with dueling architecture: $Q(s,a) = V(s;\theta_V) + (A(s,a;\theta_A) - \frac{1}{|A|}\sum_{a'} A(s,a';\theta_A))$ ✦ (Dueling network architecture)
✦ Replace linear layers with noisy layers: $y = (\mu_w + \sigma_w \odot \varepsilon)x + (\mu_b + \sigma_b \odot \varepsilon_b)$ ✦ (Noisy Networks for exploration)
Initialise target $\hat{Q}$ with $\theta^- \leftarrow \theta$
Initialise prioritised replay buffer $\mathcal{D}$ with capacity M (sum-tree)
Step counter $t \leftarrow 0$
✦ **for** $t = 1, \ldots, W$ **do** collect random-policy transitions into $\mathcal{D}$ **end for** ✦ (Replay warm-up / minimum buffer fill)
for episode $= 1, 2, \ldots, n$ **do**

Initialise s

repeat (for each step of episode)

✦ $a \leftarrow \arg\max_a Q(s, a; \theta)$ *(ε-greedy unnecessary with noisy nets)* ✦ (Noisy Networks for exploration)

Take action a; observe $r,\ s',$ done

✦ Store m-step transition $(s_t, a_t, R_t^{(m)}, s_{t+m})$ with $R^{(m)} = \sum_{k=0}^{m-1} \gamma^k r_{t+k+1}$ ✦ (n-step bootstrap returns in the buffer)

✦ Sample minibatch $j \sim P(j) \propto |\delta_j|^{\alpha_{\text{per}}}$ from $\mathcal{D}$; compute IS weights $w_j = (N\, P_j)^{-\beta_{\text{per}}}$ ✦ (Prioritized Experience Replay (PER))

for each j **do**

✦ $a_j^* \leftarrow \arg\max_{a'} Q(s'_j, a'; \theta)$; $y_j \leftarrow R_j^{(m)} + \gamma^m (1 - \text{done}_j)\, \hat{Q}(s'_j, a_j^*; \theta^-)$ *(Double DQN + m-step)* ✦ (Double DQN / n-step bootstrap returns in the buffer)

end for

✦ $\mathcal{L}(\theta) \leftarrow \frac{1}{B} \sum_j w_j \big(y_j - Q(s_j, a_j; \theta) \big)^2$ *(IS-weighted loss)* ✦ (Prioritized Experience Replay (PER))

✦ Update priorities $|\delta_j|$ in $\mathcal{D}$ ✦ (Prioritized Experience Replay (PER))

✦ *(Full Rainbow: all above + distributional C51/QR-DQN head replacing scalar Q)* ✦ (Distributional RL — C51 / QR-DQN / IQN / Rainbow)

$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$; $\quad s \leftarrow s'$; $\quad t \leftarrow t + 1$

if $t \bmod C = 0$ **then** $\theta^- \leftarrow \theta$

until done = **true**

end for

✦✦✦

✦ ✦ ✦

1.7 Deep RL — Policy Gradient Methods

✦ ✦ ✦

1.7.1 What Is the Vanilla Policy Gradient Loss?

All policy gradient methods share a common ancestor: the **policy gradient theorem**, which gives the gradient of the expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[R]$ with respect to the policy parameters:

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\; \Psi_t\right]$$

The **vanilla policy gradient loss** is the objective whose gradient recovers this formula:

$$\boxed{\mathcal{L}^{\text{VPG}}(\theta) \;=\; -\,\mathbb{E}\left[\sum_{t} \log \pi_\theta(a_t \mid s_t)\; \Psi_t\right]}$$

where Ψ_t is a scalar **signal** that tells the gradient *how good* action a_t was. Different choices of Ψ_t yield different algorithms while preserving the same loss structure:

Ψ_t	Algorithm
G_t (full Monte-Carlo return)	REINFORCE
$G_t - b(s_t)$ (baseline-subtracted return)	REINFORCE with baseline
$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (TD error)	Actor-Critic
$A_t = Q(s_t, a_t) - V(s_t)$ (advantage)	A2C / A3C

Intuition: increase the log-probability of actions that led to above-average outcomes; decrease it for below-average ones. The vanilla loss implements this idea in its simplest form — no surrogate objectives, no clipping, no entropy augmentation, no importance-sampling correction.

Methods that **depart** from this vanilla template each replace or augment the objective in a fundamental way: PPO introduces a clipped importance-sampling surrogate, SAC adds a maximum-entropy bonus to the reward, DDPG bypasses stochastic log-probabilities entirely via the deterministic policy gradient, and CQL adds a conservative regulariser for offline learning.

✦ Vanilla Policy Gradient

1.7.2 REINFORCE (Monte Carlo Policy Gradient)

*Model-free, on-policy, Monte Carlo policy gradient; learns from **complete** episodes. Instead of learning Q/V, directly parameterize and optimize the **policy** $\pi(a \mid s;\theta)$.*

Notation: - θ — policy network weights (*not* a convergence threshold; learnable parameters of π) - $J(\theta)$ — performance objective: expected total discounted return under policy $\pi(\cdot;\theta)$ - G_t — discounted return from step t to episode end: $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$ - T — terminal time step of the episode - $b(s_t)$ — baseline (any function of s_t, e.g. $V(s_t)$); subtracted to reduce gradient variance

Introduction: Model-free, on-policy ***Monte Carlo policy gradient*** — directly parameterises and optimises the policy $\pi(a \mid s;\theta)$ rather than a value function. Learns from ***complete*** episodes. *Idea:* the gradient of expected return is $\nabla J(\theta) = \mathbb{E}\big[\sum_t G_t \nabla \log \pi(a_t \mid s_t;\theta)\big]$ — increase the log-probability of actions weighted by the return that followed them. *Why it helps:* works natively for continuous and stochastic policies, where $\max_a Q$ is intractable; learns stochastic policies in their own right. *Cost:* high variance — typically reduced with a learned baseline (see Actor–Critic).

Algorithm:

Input: policy network $\pi(a \mid s;\theta)$, learning rate α, discount γ, episodes n

1. Initialize policy network $\pi(a \mid s;\theta)$ with random weights θ
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - **— Generate full episode using current π —**

     $$s_0,\, a_0,\, r_1,\ s_1,\, a_1,\, r_2,\ \ldots,\ s_{T-1},\, a_{T-1},\, r_T \quad \text{(run until terminal)}$$

   - **— Compute returns —**
   - For $t = 0, 1, \ldots, T-1$:

     $$G_t = \sum_{k=0}^{T-t-1} \gamma^k\, r_{t+k+1}$$

   - **— Policy gradient update —**

     $$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \gamma^t\, G_t\, \nabla_\theta \log \pi(a_t \mid s_t;\theta)$$

3. Return $\pi(\cdot \mid \cdot;\theta)$

✦ ✦ ✦

Policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi\left[\sum_t \gamma^t\, G_t\, \nabla_\theta \log \pi(a_t \mid s_t;\theta)\right]$$

Intuition:

- G_t high $\Rightarrow$ increase $\log \pi(a_t \mid s_t; \theta) \Rightarrow$ make a_t **more likely**
- G_t low $\Rightarrow$ decrease $\log \pi(a_t \mid s_t; \theta) \Rightarrow$ make a_t **less likely**

Notes:

- Must wait for a **complete** episode before updating (like MC) — cannot learn online.
- **High variance:** G_t can vary wildly across episodes $\Rightarrow$ noisy gradient estimates.
- Common fix — subtract a **baseline** $b(s_t)$ from G_t (reduces variance without adding bias):

$$\theta \leftarrow \theta + \alpha \sum_t \gamma^t \left(G_t - b(s_t)\right) \nabla_\theta \log \pi(a_t \mid s_t; \theta)$$

  A common baseline is $V(s_t) \Rightarrow$ this leads to **Actor-Critic** methods (see below).
- Outputs a **stochastic** policy $\pi(a \mid s; \theta)$, unlike Q-learning which outputs a deterministic policy.

Additional Known Engineering Tricks

- **Baseline subtraction** — replace G_t in the gradient with $G_t - b(s_t)$ where b is any function of state (commonly $b(s) = V(s)$ from a learned critic). *Why it helps:* $\mathbb{E}_a[b(s)\nabla \log \pi(a \mid s)] = 0$, so subtracting a baseline reduces variance *without adding bias* — by far the most impactful single trick for REINFORCE. *Interaction:* learning $b = V$ alongside π gives Actor-Critic — see Algorithm 4.2.
- **Return standardisation (whitening)** — within each episode (or batch of episodes), shift and scale: $G_t \leftarrow (G_t - \mu_G)/(\sigma_G + 10^{-8})$. *Why it helps:* removes the dependence on absolute reward scale and stabilises the gradient magnitude for Adam/SGD; the learning rate becomes scale-invariant. *Interaction:* using *both* whitening and a learned baseline subtracts the mean twice — usually pick one; whitening is the cheap drop-in if no critic is trained.
- **Adam with low learning rate ($10^{-4} - 10^{-3}$)** — REINFORCE's gradient variance is enormous, so a small adaptive step size is essential. *Why it helps:* Adam's per-parameter normalisation hides much of the gradient variance from the optimiser, allowing the small effective step to still make progress.
- **Global-norm gradient clipping** — if $\|\nabla\theta\| > c$, scale to norm c (typical $c \in [0.5, 5]$). *Why it helps:* a single high-return outlier episode can blow up the gradient; clipping caps the damage one bad batch can do.

Algorithm 4.1 REINFORCE (Monte Carlo policy gradient) — Raw Pseudocode (without additional tricks) *[Williams, 1992; Sutton & Barto, §13.3]*

Here is the pseudocode for the REINFORCE core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: policy network $\pi(a \mid s; \theta)$, learning rate α, discount γ, episodes n
Initialize θ randomly
for episode $= 1, 2, \ldots, n$ **do**
  Generate an episode using π_θ: $s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T$
  for $t = 0, 1, \ldots, T-1$ **do**

$$G_t \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

$\theta \leftarrow \theta + \alpha\, \gamma^t\, G_t\, \nabla_\theta \log \pi(a_t \mid s_t;\, \theta)$
end for
end for

Algorithm 4.1 REINFORCE (Monte Carlo policy gradient) — Pseudocode with Additions (Engineering Tricks) *[Williams, 1992; Sutton & Barto, §13.3]*

Here is the pseudocode for the REINFORCE algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: policy network $\pi(a \mid s;\, \theta)$, ✦ Adam optimiser with $\alpha \in [10^{-4},\, 10^{-3}]$ ✦ (Adam with low learning rate $(10^{-4} - 10^{-3})$), discount γ, episodes n, ✦ gradient clip norm c ✦ (Global-norm gradient clipping)
Initialise θ randomly
✦ *(Optionally initialise a baseline $b(s)$ — e.g., a learned $V(s; w)$)* ✦ (Baseline subtraction)
for episode $= 1, 2, \ldots, n$ **do**
Generate an episode using π_θ: $s_0, a_0, r_1,\ s_1, a_1, r_2,\ \ldots,\ s_{T-1}, a_{T-1}, r_T$
for $t = 0, 1, \ldots, T-1$ **do**
$G_t \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1}\, r_k$
end for
✦ Standardise returns: $G_t \leftarrow (G_t - \mu_G)/(\sigma_G + 10^{-8})$ over $\{G_0, \ldots, G_{T-1}\}$ ✦ (Return standardisation (whitening))
for $t = 0, 1, \ldots, T-1$ **do**
✦ $\theta \leftarrow \theta + \alpha\, \gamma^t\, (G_t - b(s_t))\, \nabla_\theta \log \pi(a_t \mid s_t;\, \theta)$ *(baseline-subtracted gradient)* ✦ (Baseline subtraction)
end for
✦ **if** $\|\nabla\theta\| > c$ **then** $\nabla\theta \leftarrow c \cdot \nabla\theta / \|\nabla\theta\|$ ✦ (Global-norm gradient clipping)
✦ Apply Adam step ✦ (Adam with low learning rate $(10^{-4} - 10^{-3})$)
end for

✦✦✦

1.7.3 Actor-Critic (AC)

Notation: - w — critic (value network) weights - α_θ — actor learning rate - α_w — critic learning rate - G_t — discounted return from step t (same as REINFORCE) - γ^t — discounting applied to the policy gradient at time t

Introduction: Combines a policy gradient (***Actor***, $\pi(a \mid s;\theta)$) with a learned value function (***Critic***, $V(s;w)$). The Critic's TD error $\delta = r + \gamma V(s';w) - V(s;w)$ replaces the Monte Carlo return G_t as the policy-gradient signal.

- **Actor:** policy network $\pi(a \mid s;\theta)$ — decides which action to take
- **Critic:** value network $V(s;w)$ — evaluates how good the current state is

Idea: let the Actor choose actions and the Critic score the resulting states; use the Critic's verdict to push the Actor's log-probabilities. *Why it helps over REINFORCE:* much lower variance (TD error / advantage instead of full return), and updates can happen ***online*** at every step — no need to wait for episode termination.

Algorithm:

Input: actor rate α_θ, critic rate α_w, discount γ, episodes n

1. Initialize Actor $\pi(a \mid s;\theta)$ and Critic $V(s;w)$ with random weights
2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:
   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - $a \sim \pi(\cdot \mid s;\theta)$ *(Actor chooses action)*
     - Take action a, observe $r,\ s'$
     - **TD error (Critic's evaluation):**

       $$\delta \leftarrow r + \gamma V(s';w) - V(s;w)$$

       ($\delta > 0$: outcome was better than expected; $\delta < 0$: worse than expected)
     - **Update Critic** *(minimize TD error)*:

       $$w \leftarrow w + \alpha_w \, \delta \, \nabla_w V(s;w)$$

     - **Update Actor** *(policy gradient with δ as advantage)*:

       $$\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta \, \nabla_\theta \log \pi(a \mid s;\theta)$$

       (δ replaces G_t from REINFORCE)
     - $s \leftarrow s'$
   - Until s is terminal
3. Return $\pi(\cdot \mid \cdot;\theta)$

✦ ✦ ✦

Notes:

- Can update **every step** (online), not just at episode end.
- The TD error $\delta = r + \gamma V(s') - V(s)$ estimates the **advantage** $A(s,a)$ = "how much better was this action than what I expected from this state?"

- **Lower variance** than REINFORCE (δ is less noisy than G_t), but introduces **bias** (because $V(s'; w)$ is an approximation).
- Two separate networks: Actor (θ) and Critic (w), each with its own learning rate. They can optionally share lower layers for feature extraction.

Additional Known Engineering Tricks

- **Shared trunk + two heads** — let actor and critic share the bulk of the network (conv stack for images, MLP body for state vectors), branching only at the last layer into a policy head and a value head. *Why it helps:* one feature extractor is forced to encode information useful for both objectives, which acts as a regulariser; halves parameter count and forward-pass cost. *Interaction:* if the actor and critic learning rates differ by orders of magnitude, the shared trunk's gradient can be dominated by one head — fix with separate optimisers per head or per-head loss-scale weights.
- **Target value network for the critic** — keep a slow-moving copy $V(s; w^-)$ and use $\delta = r + \gamma V(s'; w^-) - V(s; w)$ instead of $V(s'; w)$. *Why it helps:* same logic as DQN+TN — prevents the critic's bootstrap target from chasing its own moving estimate; particularly important when the actor and critic share weights, because actor gradients perturb the critic indirectly.
- **n-step / multi-step bootstrapped returns** — replace the one-step δ with $\delta^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n}; w) - V(s_t; w)$. *Why it helps:* the same bias-variance dial as n-step TD, but inside the actor's advantage signal; recovers the full episode return at $n = T$ (REINFORCE-style) and one-step TD at $n = 1$.
- **Entropy regularisation** $\beta\, H(\pi(\cdot \mid s))$ — add a small positive bonus on policy entropy to the actor's objective. *Why it helps:* prevents premature collapse of the stochastic policy onto a single action; on small problems the basic AC can converge to a near-deterministic suboptimal policy quickly without it. *Interaction:* this is what A2C/A3C and PPO inherit as a non-negotiable component.
- **Eligibility traces — AC(λ)** — maintain traces e_θ, e_w on both networks and update with δ scaled by the trace. *Why it helps:* offline-equivalent of the λ-return for both actor and critic, with $O(1)$ amortised cost per step.
- **Pop-Art / adaptive reward normalisation** *(van Hasselt et al., 2016)* — predict normalised values $\tilde{V}$ and adapt the output layer's affine to preserve V's scale as the running stats change. *Why it helps:* lets the critic learn across tasks with wildly different reward scales without retuning the learning rate; the actor sees a stable advantage signal regardless.

Algorithm 4.2 One-step Actor–Critic — Raw Pseudocode (without additional tricks) *[Sutton & Barto, §13.5]*

Here is the pseudocode for the Actor–Critic core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: actor lr α_θ, critic lr α_w, discount γ, episodes n
Initialize policy parameters θ and value parameters w randomly
for episode $= 1, 2, \ldots, n$ **do**

Initialize s
$I \leftarrow 1$
repeat (for each step of episode)
Sample $a \sim \pi(\cdot \mid s; \theta)$
Take action a; observe $r,\ s'$
$\delta \leftarrow r + \gamma\, V(s'; w) - V(s; w)$ *(TD error / advantage)*
$w \leftarrow w + \alpha_w\, \delta\, \nabla_w V(s; w)$
$\theta \leftarrow \theta + \alpha_\theta\, I\, \delta\, \nabla_\theta \log \pi(a \mid s; \theta)$
$I \leftarrow \gamma\, I$
$s \leftarrow s'$
until s is terminal
end for

Algorithm 4.2 One-step Actor–Critic — Pseudocode with Additions (Engineering Tricks) *[Sutton & Barto, §13.5]*

Here is the pseudocode for the Actor–Critic algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: actor lr α_θ, critic lr α_w, discount γ, ✦ trace decay λ ✦ (Eligibility traces — AC(λ)), ✦ entropy coefficient β ✦ (Entropy regularisation), ✦ bootstrap steps m ✦ (n-step / multi-step bootstrapped returns), episodes n
✦ Initialise shared-trunk network with actor head $\pi(\cdot \mid s; \theta)$ and critic head $V(s; w)$ ✦ (Shared trunk + two heads)
✦ Initialise target critic $V(s; w^-)$ with $w^- \leftarrow w$ ✦ (Target value network for the critic)
✦ Initialise Pop-Art running stats (μ_r, σ_r) ✦ (Pop-Art / adaptive reward normalisation)
for episode $= 1, 2, \ldots, n$ **do**
Initialise s; $I \leftarrow 1$
✦ $e_\theta \leftarrow 0,\ e_w \leftarrow 0$ *(eligibility traces)* ✦ (Eligibility traces — AC(λ))
repeat (for each step of episode)
Sample $a \sim \pi(\cdot \mid s; \theta)$
Take action a; observe $r,\ s'$
✦ $r \leftarrow (r - \mu_r)/\sigma_r$; update running stats *(Pop-Art)* ✦ (Pop-Art / adaptive reward normalisation)
✦ $\delta^{(m)} \leftarrow \sum_{k=0}^{m-1} \gamma^k r_{t+k+1} + \gamma^m V(s_{t+m}; w^-) - V(s_t; w)$ *(m-step TD error via target critic)* ✦ (n-step / multi-step bootstrapped returns / Target value network for the critic)
✦ $e_w \leftarrow \gamma\lambda\, e_w + \nabla_w V(s; w);\ \ w \leftarrow w + \alpha_w\, \delta^{(m)}\, e_w$ ✦ (Eligibility traces — AC(λ))
✦ $e_\theta \leftarrow \gamma\lambda\, e_\theta + I\, \nabla_\theta \log \pi(a \mid s; \theta)$ ✦ (Eligibility traces — AC(λ))
✦ $\theta \leftarrow \theta + \alpha_\theta\, \delta^{(m)}\, e_\theta + \beta\, \nabla_\theta H(\pi(\cdot \mid s))$ *(+ entropy bonus)* ✦ (Eligibility traces — AC(λ) / Entropy regularisation)
$I \leftarrow \gamma\, I;\ \ s \leftarrow s'$
until s is terminal
end for

✦✦✦

1.7.4 A2C (Advantage Actor-Critic)

Notation: - K — number of parallel environment workers - n — rollout length (steps each worker collects before a gradient update) - β — entropy regularisation coefficient - $H(\pi(\cdot \mid s;\theta))$ — policy entropy at state s: $-\sum_a \pi(a \mid s)\log\pi(a \mid s)$; higher ⇒ more uniform distribution ⇒ more exploration - Adv_i^k — advantage estimate for worker k at step i: $R - V(s_i^k; w)$

Same as Actor-Critic above: - w — critic (value network) weights - α_θ — actor learning rate - α_w — critic learning rate

Introduction: Synchronous advantage actor–critic. Runs K parallel environment workers under a shared Actor and Critic; ***all workers finish their n-step rollouts*** before a single combined gradient update is applied. *Idea:* use the n-step advantage $\mathrm{Adv}_i = R_i - V(s_i; w)$ (lower variance than the MC return, less biased than the 1-step TD) to drive the policy gradient, and rely on parallel rollouts in different states for decorrelation. *Why it helps:* batches of decorrelated experiences stabilise training (no replay buffer needed) and parallelism keeps GPU utilisation high; in practice often matches A3C while being simpler and avoiding stale-gradient issues.

Algorithm:

Input: α_θ, α_w, γ, rollout length n, parallel workers K, entropy coefficient β

1. Initialize shared Actor $\pi(a \mid s;\theta)$ and shared Critic $V(s;w)$

   Launch K parallel environment copies: $\mathrm{env}_1, \mathrm{env}_2, \ldots, \mathrm{env}_K$

2. **Repeat** until convergence *(outer loop — updates)*:

   - **— Each worker k collects n steps in parallel (synchronously) —**
   - For each worker $k = 1, \ldots, K$ and step $i = 1, \ldots, n$:
     - $a_i^k \sim \pi(\cdot \mid s_i^k;\theta)$; take a_i^k in env_k, observe $r_i^k,\ s_i'^k$
   - **— Compute n-step returns and advantages —**
   - For each worker k:

$$R \leftarrow \begin{cases} 0 & \text{if } s_n^k \text{ is terminal} \\ V(s_n^k; w) & \text{otherwise (bootstrap from Critic)} \end{cases}$$

     - For $i = n{-}1, \ldots, 0$ (backwards): $\quad R \leftarrow r_{i+1}^k + \gamma R; \quad \mathrm{Adv}_i^k \leftarrow R - V(s_i^k; w)$
   - **— Aggregate losses across all K workers —**

$$\mathcal{L}_{\text{actor}}(\theta) = -\frac{1}{K}\sum_k \sum_i \mathrm{Adv}_i^k \cdot \log\pi(a_i^k \mid s_i^k;\theta) \;-\; \beta\, H(\pi(\cdot \mid s_i^k;\theta))$$

$$\mathcal{L}_{\text{critic}}(w) = \frac{1}{K}\sum_k \sum_i \big(R - V(s_i^k; w)\big)^2$$

   - **— Single synchronous gradient update —**

$$\theta \leftarrow \theta - \alpha_\theta \nabla_\theta \mathcal{L}_{\text{actor}}, \qquad w \leftarrow w - \alpha_w \nabla_w \mathcal{L}_{\text{critic}}$$

3. Return $\pi(\cdot \mid \cdot; \theta)$

✦ ✦ ✦

Notes:

- **Synchronous:** all K workers finish their rollouts, then ONE gradient update happens. Simpler and more stable than A3C. No stale gradients.
- **Entropy bonus** $\beta H(\pi)$ encourages exploration by penalizing overly deterministic policies: $H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$. Higher entropy ⇒ more uniform ⇒ more exploration.
- K parallel envs ⇒ each update uses $K \times n$ transitions ⇒ lower-variance gradient estimates.
- n-step returns combine TD bootstrapping with multi-step actual rewards (like n-step TD).
- In practice, actor and critic often share a neural network body with two output heads.

Additional Known Engineering Tricks

- **Generalised Advantage Estimation — GAE(λ)** *(Schulman et al., 2016)* — replace $R - V(s)$ with $\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}$. *Why it helps:* the same bias-variance dial as λ-returns, applied directly to the advantage; $\lambda \in [0.9, 0.97]$ is the empirical sweet spot in continuous-control benchmarks.
- **Orthogonal weight initialisation, gain $\sqrt{2}$** *(Saxe et al., 2014; ICLR-blog standard for PPO/A2C)* — orthogonal init on hidden layers, gain 0.01 on the policy logits head, gain 1.0 on the value head. *Why it helps:* the tiny gain on the logits keeps the initial policy nearly uniform (high entropy) so exploration starts broad; orthogonal weights preserve gradient norm across depth.
- **Observation normalisation with running mean/std** — maintain online estimates of per-feature (μ, σ) across all workers and feed $(s - \mu)/\sigma$ to the network. *Why it helps:* removes the dependence on the raw observation scale; critical for MuJoCo-style proprioceptive inputs whose features differ by orders of magnitude.
- **Advantage normalisation per batch** — $\hat{A} \leftarrow (\hat{A} - \mu_A)/(\sigma_A + 10^{-8})$. *Why it helps:* the loss is then scale-invariant in advantage units; this is the *single* most commonly-cited 'magic' trick separating a working A2C/PPO from a broken one.
- **Global-norm gradient clipping (typically 0.5)** — clip $\|\nabla(\theta, w)\|_2$ at a fixed norm before the optimiser step. *Why it helps:* one anomalous rollout (e.g., a sudden mega-reward) cannot blow up the parameters in a single step.
- **Frame stacking + frame skipping (Atari)** — inherits the four-frame stack and action-repeat-4 pre-processing from DQN. *Why it helps:* provides a velocity signal in pixel input and cuts per-decision compute, exactly as in value-based methods.

Algorithm 4.3 A2C (synchronous advantage actor–critic) — Raw Pseudocode (without additional tricks) *[Mnih et al., 2016 (synchronous variant)]*

Here is the pseudocode for the A2C core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: α_θ, α_w, γ, rollout length n, parallel workers K, entropy coefficient β
Initialize global parameters θ and w

repeat until convergence
  // Each worker collects n steps with current π_θ
  for $k = 1, 2, \dots, K$ in parallel **do**
    for $i = 0, 1, \dots, n-1$ **do**
      Sample $a_i^k \sim \pi(\cdot \mid s_i^k;\, \theta)$; observe $r_{i+1}^k,\ s_{i+1}^k$
    end for
  end for
  // Compute n-step returns and advantages
  for $k = 1, 2, \dots, K$ **do**
    $R \leftarrow 0$ if s_n^k terminal else $V(s_n^k;\, w)$
    for $i = n-1$ **down to** 0 **do**
      $R \leftarrow r_{i+1}^k + \gamma\, R$
      $\hat{A}_i^k \leftarrow R - V(s_i^k;\, w); \quad R_i^k \leftarrow R$
    end for
  end for
  // Joint update over all Kn samples
  $\mathcal{L}_\theta \leftarrow -\dfrac{1}{Kn} \sum_{k,i} \big[\log \pi(a_i^k \mid s_i^k;\, \theta)\, \hat{A}_i^k + \beta\, \mathcal{H}\big(\pi(\cdot \mid s_i^k;\, \theta)\big) \big]$
  $\mathcal{L}_w \leftarrow \dfrac{1}{Kn} \sum_{k,i} \big(R_i^k - V(s_i^k;\, w) \big)^2$
  $\theta \leftarrow \theta - \alpha_\theta\, \nabla_\theta\, \mathcal{L}_\theta; \quad w \leftarrow w - \alpha_w\, \nabla_w\, \mathcal{L}_w$
end repeat

Algorithm 4.3 A2C (synchronous advantage actor–critic) — Pseudocode with Additions (Engineering Tricks) *[Mnih et al., 2016 (synchronous variant)]*

Here is the pseudocode for the A2C algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: α_θ, α_w, γ, rollout length n, parallel workers K, entropy coefficient β, ✦ GAE λ ✦ (Generalised Advantage Estimation — GAE(λ)), ✦ gradient clip norm $c = 0.5$ ✦ (Global-norm gradient clipping (typically 0.5))
✦ Initialise shared Actor and Critic with orthogonal weights (gain $\sqrt{2}$ hidden, gain 0.01 policy head, gain 1.0 value head) ✦ (Orthogonal weight initialisation, gain $\sqrt{2}$)
✦ Initialise running observation statistics (μ_s, σ_s) across all workers ✦ (Observation normalisation with running mean/std)
repeat until convergence
  for $k = 1, 2, \dots, K$ in parallel **do**
    for $i = 0, 1, \dots, n-1$ **do**
      ✦ Pre-process: frame-stack + frame-skip (Atari) or normalise $s \leftarrow (s - \mu_s)/\sigma_s$ ✦ (Frame stacking + frame skipping (Atari) / Observation normalisation with running mean/std)
      Sample $a_i^k \sim \pi(\cdot \mid s_i^k;\, \theta)$; observe $r_{i+1}^k,\ s_{i+1}^k$
    end for
  end for
  ✦ *// Compute GAE advantages* ✦ (Generalised Advantage Estimation — GAE(λ))
  for $k = 1, 2, \dots, K$ **do**

$\delta_i^k \leftarrow r_{i+1}^k + \gamma\, V(s_{i+1}^k;\, w) - V(s_i^k;\, w)$

✦ $\hat{A}_i^k \leftarrow \sum_{l \geq 0} (\gamma\lambda)^l\, \delta_{i+l}^k$ ✦ (Generalised Advantage Estimation — GAE(λ))

$R_i^k \leftarrow \hat{A}_i^k + V(s_i^k;\, w)$

end for

✦ $\hat{A} \leftarrow (\hat{A} - \mu_{\hat{A}})/(\sigma_{\hat{A}} + 10^{-8})$ ✦ (Advantage normalisation per batch)

$\mathcal{L}_\theta \leftarrow -\dfrac{1}{Kn} \sum_{k,i} \big[\log \pi(a_i^k \mid s_i^k;\, \theta)\, \hat{A}_i^k + \beta\, \mathcal{H}\big(\pi(\cdot \mid s_i^k;\, \theta)\big) \big]$

$\mathcal{L}_w \leftarrow \dfrac{1}{Kn} \sum_{k,i} \big(R_i^k - V(s_i^k;\, w) \big)^2$

✦ **if** $\|\nabla(\theta, w)\|_2 > c$ **then** rescale to norm c ✦ (Global-norm gradient clipping (typically 0.5))

$\theta \leftarrow \theta - \alpha_\theta\, \nabla_\theta\, \mathcal{L}_\theta; \quad w \leftarrow w - \alpha_w\, \nabla_w\, \mathcal{L}_w$

end repeat

✦✦✦

1.7.5 A3C (Asynchronous Advantage Actor-Critic)

Notation: - θ_{local}, w_{local} — worker-local copies of global θ, w; pulled at the start of each rollout - $d\theta$, dw — gradient increments computed locally and asynchronously pushed to global params - K — number of parallel asynchronous workers - n — rollout length (steps each worker collects before pushing $d\theta, dw$) - β — entropy regularisation coefficient in the actor loss - $H(\pi(\cdot \mid s; \theta_{\text{local}})) = -\sum_a \pi(a \mid s; \theta_{\text{local}}) \log \pi(a \mid s; \theta_{\text{local}})$ — policy entropy under the worker's local policy - $\text{Adv}_i = R_i - V_{w_{\text{local}}}(s_i)$ — n-step advantage at step i within a rollout, with R_i the bootstrapped n-step return

Introduction: Asynchronous advantage actor–critic. K worker threads run independent environments and ***asynchronously*** push gradient updates to a shared global (θ, w) — each worker pushes its gradients independently without waiting for others. *Idea:* replace experience replay with parallelism: the diversity of states across asynchronous workers naturally decorrelates updates, and each worker uses an n-step advantage as in A2C. *Why it helps (historically):* enabled deep policy-gradient methods on CPU-only multi-core machines pre-GPU-friendly batching; now largely superseded by A2C (simpler, equally fast, no stale gradients).

Algorithm:

Input: α_θ, α_w, γ, rollout length n, parallel workers K, entropy coefficient β

1. Initialize shared (global) Actor $\pi(a \mid s; \theta)$ and Critic $V(s; w)$
2. **For each worker** $k = 1, \dots, K$ running **asynchronously** in parallel:
   - **Repeat** *(worker loop)*:
     - **Sync:** $\theta_{\text{local}} \leftarrow \theta$; $\quad w_{\text{local}} \leftarrow w$ *(pull latest global weights)*
     - **Collect** n-step rollout using $\pi(\cdot; \theta_{\text{local}})$:
       - For $i = 1, \dots, n$ (or until terminal):
         - $a_i \sim \pi(\cdot \mid s_i; \theta_{\text{local}})$; take a_i, observe r_i, s'_i
     - **Compute n-step returns and advantages:**

$$R \leftarrow \begin{cases} 0 & s_n \text{ is terminal} \\ V(s_n; w_{\text{local}}) & \text{otherwise (bootstrap)} \end{cases}$$

       - For $i = n{-}1, \dots, 0$: $\quad R \leftarrow r_{i+1} + \gamma R$; $\quad \text{Adv}_i \leftarrow R - V(s_i; w_{\text{local}})$
     - **Compute local gradients:**

$$d\theta \leftarrow \nabla_{\theta_{\text{local}}} \left[-\sum_i \text{Adv}_i \cdot \log \pi(a_i \mid s_i; \theta_{\text{local}}) \; - \; \beta \, H(\pi(\cdot \mid s_i; \theta_{\text{local}})) \right]$$

$$dw \leftarrow \nabla_{w_{\text{local}}} \left[\sum_i \big(R - V(s_i; w_{\text{local}})\big)^2 \right]$$

     - **Async push to global params** *(no lock — atomic add)*:

$$\theta \leftarrow \theta - \alpha_\theta \, d\theta, \qquad w \leftarrow w - \alpha_w \, dw$$

- Until global step count reaches maximum

3. Return $\pi(\cdot \mid \cdot; \theta)$

✦ ✦ ✦

Key differences from A2C:

- **Asynchronous:** each worker updates global params as soon as it finishes its rollout, **without waiting** for other workers. Workers run at different speeds.
- Workers may compute gradients using **stale params** (θ_{local} may be outdated by the time the gradient is applied to θ).
- No replay buffer needed: the diversity of experience from K async workers in different states naturally decorrelates updates.
- In practice, **A2C (synchronous) is often preferred** — simpler, equally fast on GPU, and avoids the stale gradient issue.

Additional Known Engineering Tricks

- **HOGWILD!-style lock-free async updates** *(Recht et al., 2011; adopted by Mnih et al., 2016)* — each worker thread reads (θ, w) without a lock and atomically adds its gradient. *Why it helps:* eliminates the global synchronisation barrier that bottlenecks A2C, so wall-clock scaling is near-linear in CPU cores. *Interaction:* relies on gradient updates being sparse / commutative-enough that occasional races don't destabilise — the entire raison d'être of A3C over A2C.

- **Worker-local optimizer state (RMSProp accumulators)** — each thread keeps its own moving-average squared-gradient buffer, but the parameters θ are global. *Why it helps:* without local optimizer state, every async write contaminates a single global accumulator with stale gradients and the adaptive step size collapses.

- **Per-worker exploration parameters** — assign each thread a *different* ε (e.g., sampled from a log-uniform distribution) or different entropy bonus, drawn once per worker lifetime. *Why it helps:* explicit diversity in the data-collecting policies replaces the experience replay that on-policy AC cannot use; the global parameter sees gradients from many exploration regimes simultaneously.

- **LSTM policy for partial observability** *(original A3C paper)* — replace the feed-forward policy head with an LSTM whose hidden state is reset at episode boundaries and carried within a rollout. *Why it helps:* lets the agent remember information beyond the current frame stack; the recurrent state is naturally per-worker. *Interaction:* the asynchronous setup makes recurrent training tractable because gradient noise is high — you cannot afford a tiny effective batch with BPTT on a single worker.

- **Atomic gradient accumulation** — push gradients via atomic-add CPU instructions rather than locking; on GPU implementations, fall back to small-batch synchronous updates because GPU memory isn't truly shared lock-freely. *Why it helps:* the CPU-friendly version of HOGWILD! that makes A3C a CPU-first algorithm; on GPUs the synchronous A2C variant is almost always preferred.

Algorithm 4.4 A3C — asynchronous advantage actor–critic (per actor-learner thread) — Raw Pseudocode (without additional tricks) *[Mnih et al., 2016]*

Here is the pseudocode for the A3C core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: global shared parameter vectors θ and ϕ; global shared counter $T = 0$
Assume thread-specific parameter vectors θ' and ϕ'
Initialize thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\phi \leftarrow 0$
  Synchronize thread-specific parameters $\theta' = \theta$ and $\phi' = \phi$
  $t_{\text{start}} \leftarrow t$
  Get state s_t
  repeat
    Perform a_t according to policy $\pi(a_t \mid s_t; \theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{\text{start}} = t_{\max}$
  $R \leftarrow \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t,\, \phi') & \text{for non-terminal } s_t \end{cases}$ *(bootstrap from last state)*
  for $i \in \{t-1, \ldots, t_{\text{start}}\}$ **do**
    $R \leftarrow r_i + \gamma\, R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i \mid s_i; \theta')\,(R - V(s_i; \phi'))$
    Accumulate gradients wrt ϕ': $d\phi \leftarrow d\phi + \partial \big(R - V(s_i; \phi')\big)^2 / \partial \phi'$
  end for
  Perform asynchronous update of θ using $d\theta$ and of ϕ using $d\phi$ HOGWILD!-style lock-free async update
until $T > T_{\max}$

Algorithm 4.4 A3C — asynchronous advantage actor–critic (per actor-learner thread) — Pseudocode with Additions (Engineering Tricks) *[Mnih et al., 2016]*

Here is the pseudocode for the A3C algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: global shared parameter vectors θ and ϕ; global shared counter $T = 0$
✦ Each worker k draws its own $\varepsilon_k \sim$ LogUniform (or distinct entropy bonus) ✦ (Per-worker exploration parameters)
✦ Each worker maintains local RMSProp accumulators (g_θ^2, g_ϕ^2) ✦ (Worker-local optimizer state (RMSProp accumulators))
Assume thread-specific parameter vectors θ' and ϕ'
Initialise thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\phi \leftarrow 0$

Synchronise thread-specific parameters $\theta' = \theta$ and $\phi' = \phi$
$t_{\text{start}} \leftarrow t$
Get state s_t
✦ Reset LSTM hidden state h at episode boundaries; carry within rollout ✦ (LSTM policy for partial observability)
repeat
✦ Perform a_t according to policy $\pi(a_t \mid s_t, h; \theta')$ *(LSTM policy)* ✦ (LSTM policy for partial observability)
Receive reward r_t and new state s_{t+1}
$t \leftarrow t + 1$
$T \leftarrow T + 1$
until terminal s_t **or** $t - t_{\text{start}} = t_{\max}$
$R \leftarrow \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t,\, \phi') & \text{for non-terminal } s_t \end{cases}$
for $i \in \{t-1, \ldots, t_{\text{start}}\}$ **do**
$R \leftarrow r_i + \gamma\, R$
Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i \mid s_i; \theta')\,(R - V(s_i; \phi'))$
Accumulate gradients wrt ϕ': $d\phi \leftarrow d\phi + \partial \big(R - V(s_i; \phi')\big)^2 / \partial \phi'$
end for
✦ Atomically add $d\theta$ to global θ and $d\phi$ to global ϕ *(lock-free HOGWILD! update via atomic CPU instructions)* ✦ (HOGWILD!-style lock-free async updates / Atomic gradient accumulation)
✦ Update worker-local RMSProp accumulators ✦ (Worker-local optimizer state (RMSProp accumulators))
until $T > T_{\max}$

✦✦✦

1.7.6 DDPG (Deep Deterministic Policy Gradient)

*Off-policy, actor-critic for **continuous action spaces**. Combines DPG with DQN tricks (target networks + replay). Like DQN but the* $\max_a Q$ *is replaced by the actor's output* $\mu(s)$*, since enumerating* a *is infeasible in* $\mathbb{R}^d$*.*

Notation: - $\mu(s;\theta^\mu)$ — **deterministic** actor (policy network); outputs a single continuous action vector, not a distribution - $Q(s,a;\theta^Q)$ — critic (state-action value network) - $\theta^{\mu-}$, θ^{Q-} — target actor and target critic weights (slow-moving copies of θ^μ, θ^Q) - τ — Polyak soft-update coefficient (e.g., $\tau = 0.005$): $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ - $\mathcal{N}_t$ — exploration noise added to actions (e.g., Ornstein-Uhlenbeck or Gaussian) - $\mathcal{D}$, B, M — replay buffer, minibatch size, capacity (same as DQN+ER)

Introduction: Off-policy actor–critic for ***continuous action spaces***. Combines the deterministic policy gradient (DPG) with DQN-style tricks (target networks + replay buffer). *Idea:* learn a ***deterministic*** actor $\mu(s;\theta^\mu)$ alongside a Q-critic $Q(s,a;\theta^Q)$. Since $\max_a Q$ is intractable in $\mathbb{R}^d$, replace it with $Q(s,\mu(s))$ and update the actor by gradient ascent on this — the chain rule pushes μ toward actions that the critic rates highly. *Why it helps:* brings DQN-style off-policy learning to continuous control, where Q-learning's arg max cannot be enumerated.

Algorithm:

Input: actor rate α_μ, critic rate α_Q, discount γ, soft-update τ, buffer capacity M, minibatch size B, episodes n

1. Initialize Actor $\mu(s;\theta^\mu)$ and Critic $Q(s,a;\theta^Q)$ with random weights

   Initialize target networks $\theta^{\mu-} \leftarrow \theta^\mu,\ \theta^{Q-} \leftarrow \theta^Q$

   Initialize replay buffer $\mathcal{D}$ with capacity M

2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:

   - Initialize s and exploration noise process $\mathcal{N}$
   - **Repeat** for each step *(inner loop — time steps)*:
     - $a \leftarrow \mu(s;\theta^\mu) + \mathcal{N}_t$ *(deterministic action + exploration noise)*
     - Take action a, observe $r,\ s',$ done
     - Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$
     - Sample B transitions $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$ uniformly from $\mathcal{D}$
     - **Compute critic target** *(uses target actor and target critic)*:

$$y_j = \begin{cases} r_j & \text{if done}_j \\ r_j + \gamma\, Q\big(s'_j,\ \mu(s'_j;\theta^{\mu-});\ \theta^{Q-}\big) & \text{otherwise} \end{cases}$$

     - **Update Critic** *(minimize squared Bellman error)*:

$$\mathcal{L}(\theta^Q) = \frac{1}{B}\sum_j \big(y_j - Q(s_j, a_j; \theta^Q)\big)^2, \qquad \theta^Q \leftarrow \theta^Q - \alpha_Q\, \nabla_{\theta^Q}\, \mathcal{L}(\theta^Q)$$

     - **Update Actor** *(deterministic policy gradient — chain rule through Q)*:

$$\nabla_{\theta^\mu} J \approx \frac{1}{B}\sum_j \nabla_a Q(s_j, a; \theta^Q)\big|_{a=\mu(s_j)} \cdot \nabla_{\theta^\mu}\mu(s_j;\theta^\mu)$$

$$\theta^\mu \leftarrow \theta^\mu + \alpha_\mu\, \nabla_{\theta^\mu} J$$

- **Soft-update target networks** *(Polyak averaging)*:

$$\theta^{Q-} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q-}, \qquad \theta^{\mu-} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu-}$$

- $s \leftarrow s'$
- Until s is terminal

3. Return $\mu(\cdot;\theta^{\mu})$

✦ ✦ ✦

Notes:

- **Continuous actions:** the $\max_{a'}$ in DQN's Bellman target is replaced by $Q(s', \mu(s';\theta^{\mu-}))$ — the actor *approximates* the argmax over continuous a.
- **Deterministic policy gradient** (Silver et al., 2014): for a deterministic μ, the policy gradient simplifies to $\nabla_a Q \cdot \nabla_{\theta^{\mu}} \mu$ (chain rule through the critic).
- **Exploration is external:** since μ is deterministic, exploration is injected via additive noise $\mathcal{N}_t$ on the action (Ornstein-Uhlenbeck for temporally correlated noise, or simple Gaussian).
- **Soft target updates:** rather than hard copies every C steps (DQN), target networks slowly track the online networks: $\theta^{-} \leftarrow \tau\theta + (1-\tau)\theta^{-}$.
- **Known weaknesses:** brittle to hyperparameters, **critic Q-value overestimation** (similar to DQN max bias). Fixed by **TD3** (twin critics + delayed actor updates + target action smoothing) and **SAC** (entropy regularisation, stochastic policy).

Additional Known Engineering Tricks

- **Layer normalisation in critic and actor** — apply LayerNorm after each hidden layer; the original DDPG paper used BatchNorm but it interacts badly with replay (running stats drift). *Why it helps:* normalises feature scales, which is critical because state vectors in continuous control span very different magnitudes per dimension.

- **Parameter-space noise for exploration** *(Plappert et al., 2017)* — perturb θ^{μ} itself with Gaussian noise (re-sampled every K steps) instead of adding $\mathcal{N}_t$ to the action. *Why it helps:* yields state-consistent exploration (the same state evokes the same perturbed action within a rollout), which acts as a primitive over time-correlated noise; combine with action-space noise via an adaptive scale schedule.

- **Tanh action squashing + reward/state scaling** — pass the actor output through tanh and rescale to $[a_{\text{low}}, a_{\text{high}}]$; centre and unit-scale rewards via running stats. *Why it helps:* keeps actions bounded without clipping (clipping kills the gradient); reward scaling means the same learning rate works across tasks with wildly different reward magnitudes.

- **TD3 extensions** *(Fujimoto et al., 2018)* — three drop-in fixes for DDPG's instability: (i) **twin critics** with target $y = r + \gamma \min_{i=1,2} Q_{\theta_i^-}(s', \mu(s';\theta^{\mu-}))$; (ii) **delayed actor & target updates** every 2 critic steps; (iii) **target action smoothing** $\tilde{a} = \mu(s';\theta^{\mu-}) + \text{clip}(\epsilon, -c, c)$, $\epsilon \sim \mathcal{N}(0, \sigma_t^2)$. *Why each helps:* twin-min combats critic overestimation; delayed actor updates wait for the critic to catch up before changing the policy; target smoothing forces the critic to be smooth in the action input, killing exploitable sharp critic peaks. *Interaction:* these are essentially *the* fix to DDPG and are almost always applied — bare DDPG is rarely used.

- **Replay warm-up with random actor** — for the first $\sim$ 10k steps, sample uniformly from the action space instead of using $\mu(s)$. *Why it helps:* the freshly-initialised critic has

uninformative Q-values; sampling random actions gives diverse coverage to bootstrap learning, similar to DQN's $\varepsilon = 1$ warm-up.

Algorithm 4.5 DDPG (deep deterministic policy gradient) — Raw Pseudocode (without additional tricks) *[Lillicrap et al., 2016]*

Here is the pseudocode for the DDPG core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: actor lr α_μ, critic lr α_Q, discount γ, soft-update τ, buffer capacity M, minibatch size B, episodes n

Initialize critic $Q(s, a;\, \theta^Q)$ and actor $\mu(s;\, \theta^\mu)$ with random weights

Initialize target networks: $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer $\mathcal{D}$ with capacity M

for episode $= 1, 2, \ldots, n$ **do**

  Initialize exploration noise process $\mathcal{N}$

  Initialize s

  repeat (for each step of episode)

    $a \leftarrow \mu(s;\, \theta^\mu) + \mathcal{N}_t$

    Take action a; observe r, s', done

    Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$

    Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$

    $y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)\, Q(s'_j,\, \mu(s'_j;\, \theta^{\mu'});\, \theta^{Q'})$

    $\mathcal{L}_Q \leftarrow \dfrac{1}{B} \sum_j \big(\, y_j - Q(s_j, a_j;\, \theta^Q)\,\big)^2$

    $\theta^Q \leftarrow \theta^Q - \alpha_Q\, \nabla_{\theta^Q}\, \mathcal{L}_Q$

    $\nabla_{\theta^\mu} J \approx \dfrac{1}{B} \sum_j \nabla_a\, Q(s_j, a;\, \theta^Q)\big|_{a=\mu(s_j;\,\theta^\mu)}\, \nabla_{\theta^\mu}\, \mu(s_j;\, \theta^\mu)$

    $\theta^\mu \leftarrow \theta^\mu + \alpha_\mu\, \nabla_{\theta^\mu} J$

    // Soft target updates (Polyak averaging)

    $\theta^{Q'} \leftarrow \tau\, \theta^Q + (1 - \tau)\, \theta^{Q'}$

    $\theta^{\mu'} \leftarrow \tau\, \theta^\mu + (1 - \tau)\, \theta^{\mu'}$

    $s \leftarrow s'$

  until s is terminal

end for

Algorithm 4.5 DDPG (deep deterministic policy gradient) — Pseudocode with Additions (Engineering Tricks) *[Lillicrap et al., 2016]*

Here is the pseudocode for the DDPG algorithm with also employing all of the additions and engineering tricks mentioned above:

Input: actor lr α_μ, critic lr α_Q, discount γ, soft-update τ, buffer capacity M, minibatch size B, episodes n, ✦ warm-up steps $W \approx 10\text{k}$ ✦ (Replay warm-up with random actor), ✦ TD3 smoothing noise σ_t, clip c ✦ (TD3 extensions)

✦ Initialise twin critics $Q_{\theta_1^Q}, Q_{\theta_2^Q}$ with LayerNorm after each hidden layer ✦ (Layer normalisation in critic and actor / TD3 extensions)

✦ Initialise actor $\mu(s;\,\theta^{\mu})$ with LayerNorm; output through tanh rescaled to $[a_{\text{low}}, a_{\text{high}}]$ ✦ (Layer normalisation in critic and actor / Tanh action squashing + reward/state scaling)
Initialise targets: $\theta_i^{Q'} \leftarrow \theta_i^{Q}$, $\theta^{\mu'} \leftarrow \theta^{\mu}$
Initialise replay buffer $\mathcal{D}$ with capacity M
✦ Maintain running reward/state statistics $(\mu_r, \sigma_r), (\mu_s, \sigma_s)$ ✦ (Tanh action squashing + reward/state scaling)
✦ **for** first W steps **do** sample $a \sim \text{Uniform}(a_{\text{low}}, a_{\text{high}})$, store in $\mathcal{D}$ **end for** ✦ (Replay warm-up with random actor)
for episode $= 1, 2, \ldots, n$ **do**
  Initialise s
  ✦ *(Optionally perturb θ^{μ} with Gaussian noise for parameter-space exploration)* ✦ (Parameter-space noise for exploration)
  repeat (for each step of episode)
    ✦ $a \leftarrow \tanh\!\big(\mu((s-\mu_s)/\sigma_s;\,\theta^{\mu})\big)\cdot \text{scale} + \mathcal{N}_t$ *(or use parameter-space noise)* ✦ (Tanh action squashing + reward/state scaling / Parameter-space noise for exploration)
    Take action a; observe $r,\ s',\ \text{done}$
    ✦ $r \leftarrow r/\sigma_r$ *(reward scaling)* ✦ (Tanh action squashing + reward/state scaling)
    Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$
    Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$
    ✦ $\tilde{a}' \leftarrow \mu(s'_j;\,\theta^{\mu'}) + \text{clip}(\epsilon,\,-c,\,c),\ \epsilon \sim \mathcal{N}(0, \sigma_t^2)$ *(target action smoothing)* ✦ (TD3 extensions)
    ✦ $y_j \leftarrow r_j + \gamma\,(1-\text{done}_j)\,\min_{i=1,2} Q_{\theta_i^{Q'}}(s'_j, \tilde{a}')$ *(twin-min target)* ✦ (TD3 extensions)
    for $i = 1, 2$ **do** $\theta_i^{Q} \leftarrow \theta_i^{Q} - \alpha_Q\,\nabla_{\theta_i^{Q}} \frac{1}{B}\sum_j (y_j - Q_{\theta_i^{Q}}(s_j, a_j))^2$ **end for**
    ✦ **if** $t \bmod 2 = 0$ **then** *(delayed actor + target updates)* ✦ (TD3 extensions)
      $\nabla_{\theta^{\mu}} J \approx \dfrac{1}{B}\sum_j \nabla_a Q_{\theta_1^{Q}}(s_j, a)\big|_{a=\mu(s_j;\,\theta^{\mu})}\,\nabla_{\theta^{\mu}}\,\mu(s_j;\,\theta^{\mu})$
      $\theta^{\mu} \leftarrow \theta^{\mu} + \alpha_{\mu}\,\nabla_{\theta^{\mu}} J$
      $\theta_i^{Q'} \leftarrow \tau\,\theta_i^{Q} + (1-\tau)\,\theta_i^{Q'}; \quad \theta^{\mu'} \leftarrow \tau\,\theta^{\mu} + (1-\tau)\,\theta^{\mu'}$
    ✦ **end if** ✦ (TD3 extensions)
    $s \leftarrow s'$
  until s is terminal
end for

✦✦✦

1.7.7 SAC (Soft Actor-Critic)

Off-policy, actor-critic for continuous actions. Builds on DDPG with: ***stochastic policy***, ***twin critics*** *(min combats overestimation, as in TD3), and a* ***maximum-entropy objective*** *that explicitly rewards exploration.*

Notation: - $\pi(a \mid s;\phi)$ — **stochastic** Gaussian actor (typically tanh-squashed): outputs $(\mu_\phi(s), \sigma_\phi(s))$; action $a = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \xi)$, $\xi \sim \mathcal{N}(0, I)$ - $Q_{\theta_1}, Q_{\theta_2}$ — **twin critics** (two independent Q-networks; using the min combats overestimation) - θ_1^-, θ_2^- — target critic weights (Polyak soft updates with coefficient τ) - α — **entropy temperature**: trades off expected return vs. policy entropy (can be tuned automatically toward a target entropy $\bar{H}$) - $H(\pi(\cdot \mid s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a \mid s)]$ — policy entropy at state s - $\mathcal{D}$, B, τ — replay buffer, minibatch size, soft-update coefficient (as in DDPG)

Introduction: Off-policy actor–critic for continuous actions. Builds on DDPG with three key changes: a ***stochastic*** (Gaussian, tanh-squashed) actor, ***twin critics*** that take the min to combat overestimation (as in TD3), and a ***maximum-entropy objective*** that explicitly rewards exploration.

Maximum-entropy objective:

$$J(\pi) = \mathbb{E}\left[\sum_t \gamma^t (r_t + \alpha\, H(\pi(\cdot \mid s_t)))\right]$$

The agent maximises reward ***plus*** *expected entropy — staying as random as possible while still solving the task. Why it helps:* the stochastic policy explores natively (no external noise needed), twin critics tame the overestimation that destabilises DDPG, and the entropy bonus prevents premature collapse to a deterministic local optimum — yielding state-of-the-art sample efficiency on continuous control.

Algorithm:

Input: actor rate α_ϕ, critic rate α_Q, discount γ, soft-update τ, temperature α, buffer capacity M, minibatch size B, episodes n

1. Initialize twin critics $Q_{\theta_1}, Q_{\theta_2}$ and stochastic actor $\pi(a \mid s;\phi)$

   Initialize target critics $\theta_1^- \leftarrow \theta_1$, $\theta_2^- \leftarrow \theta_2$

   Initialize replay buffer $\mathcal{D}$ with capacity M

2. **For** episode $= 1, 2, \ldots, n$ *(outer loop — episodes)*:

   - Initialize s
   - **Repeat** for each step *(inner loop — time steps)*:
     - $a \sim \pi(\cdot \mid s;\phi)$ *(sample from stochastic policy — exploration is built in)*
     - Take action a, observe r, s', done; store $(s, a, r, s', \text{done})$ in $\mathcal{D}$
     - Sample B transitions $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$ from $\mathcal{D}$
     - — **Critic targets (entropy-augmented, with twin min)** —
     - Sample $\tilde{a}'_j \sim \pi(\cdot \mid s'_j;\phi)$ for each j *(fresh sample from current actor)*
     -
     $$y_j = r_j + \gamma\,(1 - \text{done}_j)\Big[\min_{i=1,2} Q_{\theta_i^-}(s'_j, \tilde{a}'_j) \;-\; \alpha\, \log \pi(\tilde{a}'_j \mid s'_j;\phi)\Big]$$

- — **Update both critics** —

$$\mathcal{L}(\theta_i) = \frac{1}{B}\sum_j \big(y_j - Q_{\theta_i}(s_j, a_j)\big)^2, \quad \theta_i \leftarrow \theta_i - \alpha_Q \nabla_{\theta_i} \mathcal{L}(\theta_i), \quad i = 1, 2$$

- — **Update actor (reparameterised — gradient flows through sample)** —
- Sample $\tilde{a}_j = \tanh(\mu_\phi(s_j) + \sigma_\phi(s_j) \odot \xi_j),\ \xi_j \sim \mathcal{N}(0, I)$

$$\mathcal{L}(\phi) = \frac{1}{B}\sum_j \Big[\alpha \log \pi(\tilde{a}_j \mid s_j; \phi) \ - \ \min_{i=1,2} Q_{\theta_i}(s_j, \tilde{a}_j)\Big]$$

$$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}(\phi)$$

- **(Optional) Auto-tune temperature** α toward target entropy $\bar{H}$:

$$\mathcal{L}(\alpha) = -\alpha\, \mathbb{E}_{a\sim\pi}\big[\log \pi(a \mid s) + \bar{H}\big], \qquad \alpha \leftarrow \alpha - \alpha_\alpha \nabla_\alpha \mathcal{L}(\alpha)$$

- **Soft-update target critics:** $\theta_i^- \leftarrow \tau\theta_i + (1-\tau)\theta_i^-, \quad i = 1, 2$
- $s \leftarrow s'$

- Until s is terminal

3. Return $\pi(\cdot \mid \cdot; \phi)$

✦ ✦ ✦

Notes:

- **Stochastic policy** (unlike DDPG): exploration is intrinsic — no need for external action noise. The tanh squashes the Gaussian sample into a bounded action range.
- **Twin critics + min** (clipped double-Q, as in TD3): $\min(Q_{\theta_1}, Q_{\theta_2})$ in the target combats systematic Q-value overestimation.
- **Reparameterisation trick** ($a = \tanh(\mu + \sigma \odot \xi),\ \xi \sim \mathcal{N}$): lets gradients flow through the sampled action into the actor, instead of using the high-variance score-function estimator.
- **Entropy term** $-\alpha \log \pi$ inside the critic target and actor loss rewards stochasticity ⇒ exploration is principled, not heuristic.
- **Automatic temperature tuning** adjusts α to keep policy entropy near a target $\bar{H}$ (often $\bar{H} = -|A|$) — removes the most sensitive hyperparameter.
- In practice, **SAC is among the most reliable continuous-control algorithms** — strong sample efficiency, robust to hyperparameters.

Additional Known Engineering Tricks

- **Automatic temperature tuning** — adjust α by gradient descent on $\mathcal{L}(\alpha) = -\alpha\, \mathbb{E}[\log \pi(a \mid s) + \bar{H}]$, with target entropy $\bar{H} = -|A|$. *Why it helps:* removes SAC's single most sensitive hyperparameter — the entropy temperature now adapts to the task automatically. *Interaction:* tied to the reparameterised $\log \pi$ above; the Jacobian-corrected log-prob is what makes $\bar{H} = -|A|$ a sensible target. *Not shown in the pseudocode (treated as fixed α there); enable as an extra optimiser on a learnable* $\log \alpha$.

- **Update-to-data (UTD) ratio** $\gg 1$ — perform G gradient updates per environment step (typical $G = 1$, but $G = 20$ in REDQ; recent work goes up to $G = 20$ with extra critics). *Why it helps:* SAC's replay buffer hides huge sample efficiency that a single per-step update doesn't extract; increasing G trades wall-clock for sample efficiency at the cost of overfitting

to recent buffer contents. *Interaction:* large G amplifies overestimation, so it pairs with more critics or stronger regularisation.

- **Layer normalisation in critic** — apply LayerNorm after every critic hidden layer. *Why it helps:* essential when twin critics share the same input but must produce diverse estimates — LayerNorm decorrelates their representations and is part of why the min-of-two-Qs is informative rather than redundant.
- **Random-policy warm-up to fill the buffer** — first $\sim$10k env steps act uniformly random. *Why it helps:* exactly as in DDPG/DQN — the policy and critic start nowhere near useful, so random data gives broader bootstrap coverage than the entropy-maximising-but-still-untrained actor would.
- **Reward scaling** *(Haarnoja et al., 2018)* — multiply rewards by a fixed constant ($\sim$ 5 in MuJoCo tasks) before training. *Why it helps:* shifts the relative weight of reward vs. entropy in the objective; equivalent to picking a different fixed α, but easier to tune. *Interaction:* superseded by auto-tuned α when that trick is enabled.

Algorithm 4.6 SAC (soft actor–critic) — Raw Pseudocode (without additional tricks) *[Haarnoja et al., 2018]*

Here is the pseudocode for the SAC core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: actor lr α_ϕ, critic lr α_Q, discount γ, soft-update τ, temperature α, buffer M, minibatch B, episodes n

Initialize twin critics $Q_{\theta_1}, Q_{\theta_2}$ and actor π_ϕ

Initialize target critic weights $\bar{\theta}_1 \leftarrow \theta_1,\ \bar{\theta}_2 \leftarrow \theta_2$

Initialize replay buffer $\mathcal{D}$

for episode $= 1, 2, \ldots, n$ **do**

  Initialize s

  repeat (for each step of episode)

    Sample $a \sim \pi_\phi(\cdot \mid s)$; take action a; observe $r,\ s',$ done

    Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$

    Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$

    // Critic update: twin-min target with entropy bonus

    Sample $\tilde{a}'_j \sim \pi_\phi(\cdot \mid s'_j)$ for each j

    $y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)\,[\,\min_{i=1,2} Q_{\bar{\theta}_i}(s'_j,\, \tilde{a}'_j) - \alpha\, \log \pi_\phi(\tilde{a}'_j \mid s'_j)\,]$

    for $i = 1, 2$ **do**

      $\mathcal{L}_{Q_i} \leftarrow \dfrac{1}{B} \sum_j \big(\, y_j - Q_{\theta_i}(s_j, a_j)\,\big)^2$

      $\theta_i \leftarrow \theta_i - \alpha_Q\, \nabla_{\theta_i}\, \mathcal{L}_{Q_i}$

    end for

    // Actor update via reparameterisation: $\tilde{a}_j = f_\phi(\varepsilon_j;\, s_j)$

    Sample $\tilde{a}_j \sim \pi_\phi(\cdot \mid s_j)$ for each j

    $\mathcal{L}_\pi \leftarrow \dfrac{1}{B} \sum_j [\,\alpha\, \log \pi_\phi(\tilde{a}_j \mid s_j) - \min_{i=1,2} Q_{\theta_i}(s_j,\, \tilde{a}_j)\,]$

    $\phi \leftarrow \phi - \alpha_\phi\, \nabla_\phi\, \mathcal{L}_\pi$

// Soft target updates
$\bar{\theta}_i \leftarrow \tau\,\theta_i + (1-\tau)\,\bar{\theta}_i$ for $i = 1, 2$
$s \leftarrow s'$
until s is terminal
end for

Algorithm 4.6 SAC (soft actor–critic) — Pseudocode with Additions (Engineering Tricks) *[Haarnoja et al., 2018]*

Here is the pseudocode for the SAC algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: actor lr α_ϕ, critic lr α_Q, discount γ, soft-update τ, ✦ target entropy $\bar{H} = -|A|$ ✦ (Automatic temperature tuning), buffer M, minibatch B, episodes n, ✦ UTD ratio G ✦ (Update-to-data (UTD) ratio $\gg 1$), ✦ warm-up steps W ✦ (Random-policy warm-up to fill the buffer)
✦ Initialise twin critics $Q_{\theta_1}, Q_{\theta_2}$ with LayerNorm after each hidden layer ✦ (Layer normalisation in critic)
Initialise actor π_ϕ
Initialise target critic weights $\bar{\theta}_1 \leftarrow \theta_1$, $\bar{\theta}_2 \leftarrow \theta_2$
✦ Initialise learnable log-temperature $\log\alpha$ ✦ (Automatic temperature tuning)
Initialise replay buffer $\mathcal{D}$
✦ **for** first W steps **do** sample $a \sim$ Uniform, store in $\mathcal{D}$ **end for** ✦ (Random-policy warm-up to fill the buffer)
for episode $= 1, 2, \ldots, n$ **do**
Initialise s
repeat (for each step of episode)
Sample $a \sim \pi_\phi(\cdot \mid s)$; take action a; observe r, s', done
✦ $r \leftarrow r \cdot \text{reward_scale}$ ✦ (Reward scaling)
Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$
✦ **for** $g = 1, \ldots, G$ **do** *(multiple gradient steps per env step)* ✦ (Update-to-data (UTD) ratio $\gg 1$)
Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$
$\alpha \leftarrow \exp(\log\alpha)$
Sample $\tilde{a}'_j \sim \pi_\phi(\cdot \mid s'_j)$ for each j
$y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)\,[\min_{i=1,2} Q_{\bar{\theta}_i}(s'_j, \tilde{a}'_j) - \alpha \log \pi_\phi(\tilde{a}'_j \mid s'_j)]$
for $i = 1, 2$ **do** $\theta_i \leftarrow \theta_i - \alpha_Q \nabla_{\theta_i} \frac{1}{B} \sum_j (y_j - Q_{\theta_i}(s_j, a_j))^2$ **end for**
Sample $\tilde{a}_j \sim \pi_\phi(\cdot \mid s_j)$ for each j
$\mathcal{L}_\pi \leftarrow \frac{1}{B} \sum_j [\alpha \log \pi_\phi(\tilde{a}_j \mid s_j) - \min_{i=1,2} Q_{\theta_i}(s_j, \tilde{a}_j)]$
$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}_\pi$
✦ $\log\alpha \leftarrow \log\alpha - \alpha_\alpha \nabla_{\log\alpha}(-\alpha\,(\log \pi_\phi(\tilde{a}_j \mid s_j) + \bar{H}))$ *(auto-tune temperature)* ✦ (Automatic temperature tuning)
$\bar{\theta}_i \leftarrow \tau\,\theta_i + (1-\tau)\,\bar{\theta}_i$ for $i = 1, 2$
✦ **end for** ✦ (Update-to-data (UTD) ratio $\gg 1$)

$s \leftarrow s'$
until s is terminal
end for

✦✦✦

1.7.8 PPO (Proximal Policy Optimization)

*Model-free, on-policy, policy gradient; updates from **fixed-length rollouts** (no need for full episodes). The de-facto default for deep RL — **simple, stable, sample-efficient**. Solves the core trust-region problem (TRPO) with a much simpler **clipped surrogate objective** that prevents destructively large policy updates.*

Notation: - π_θ, $\pi_{\theta_{\text{old}}}$ — current and previous policies (rollouts are collected with $\pi_{\theta_{\text{old}}}$, the fixed snapshot at the start of the update) - $r_t(\theta) = \dfrac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ — **importance ratio** between new and old policies - $\hat{A}_t$ — advantage `estimate` at step t (typically **GAE** — Generalized Advantage Estimation) - ϵ — **clip range** (e.g., 0.2): how far $r_t(\theta)$ is allowed to drift from 1 - K — number of optimisation epochs per data batch (each batch reused K times) - $TD(0)$ (TD error δ_t) – the one-step estimate, low variance & high bias $A_t \approx r_t + \gamma\, V(s_{t+1}) - V(s_t)$ - MC – Monte Carlo, the full-trajectory estimate, low bias & high variance $A_t \approx (r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots) - V(s_t)$

- λ — GAE smoothing parameter $\in [0, 1]$ interpolates (tradeoff) between $TD(0)$ (bias) and MC (variance) Same as `A2C` above:
- β — entropy regularisation coefficient added to PPO's clipped actor loss to keep π_θ stochastic
- $H(\pi(\cdot \mid s;\theta)) = -\sum_a \pi(a \mid s;\theta) \log \pi(a \mid s;\theta)$ — policy entropy at state s; higher $\Rightarrow$ more uniform $\pi_\theta \Rightarrow$ more exploration
- K_{workers} — number of parallel environment workers each rolling out under $\pi_{\theta_{\text{old}}}$ (renamed from A2C's K to disambiguate from PPO's K epochs over the collected rollout)
- n — rollout length per worker before each PPO update phase (so the per-update batch contains $K_{\text{workers}} \cdot n$ transitions)

Introduction:

Model-free, on-policy policy gradient that updates from ***fixed-length rollouts*** (no need for full episodes). The de-facto default for deep RL — ***simple, stable, sample-efficient***. *Idea:* solve the core trust-region problem of TRPO with a much simpler ***clipped surrogate objective*** that prevents destructively large policy updates:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \ \text{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\,\hat{A}_t\big)\Big]$$

*Caps the incentive to push the importance ratio $r_t(\theta) = \pi_\theta / \pi_{\theta_{old}}$ beyond $1 \pm \epsilon$: **the min with the clipped term removes the gradient** as soon as the policy moves too far in either direction. Why it helps:* preserves the "small step" property that makes TRPO reliable while replacing its constrained optimisation with a single scalar clip — robust, fewer hyperparameters, and reusable data via several epochs (K) per rollout.

Algorithm:

Input: α_θ, α_w, γ, GAE λ, clip ϵ, epochs K, minibatch size B, workers K_{workers}, rollout length n, entropy coefficient β

1. Initialize Actor π_θ and Critic V_w
2. **Repeat** until convergence *(outer loop — updates)*:
    - **— Collect rollout with the *current* policy $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ —**

- For each worker $k = 1, \dots, K_{\text{workers}}$, collect n steps using $\pi_{\theta_{\text{old}}}$:
  - Store transitions $(s_t, a_t, r_t, s_{t+1}, \log \pi_{\theta_{\text{old}}}(a_t \mid s_t), V_w(s_t))$
- — **Compute advantages via GAE(λ) and returns** —
- For $t = n{-}1, \dots, 0$ (backwards):

$$\delta_t = r_t + \gamma\, V_w(s_{t+1}) - V_w(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda\, \hat{A}_{t+1}$$

$$\hat{R}_t = \hat{A}_t + V_w(s_t) \quad \text{(target for critic)}$$

- (Optionally) normalise $\hat{A}_t$ across the batch: $\hat{A}_t \leftarrow (\hat{A}_t - \mu_A)/\sigma_A$
- — **Optimise for K epochs over the *same* batch** —
- **For** epoch $= 1, \dots, K$:
  - Shuffle and split batch into minibatches of size B
  - For each minibatch:
    - Compute ratio: $r_t(\theta) = \exp\big(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\big)$
    - **Clipped policy loss:**

$$\mathcal{L}^{\text{CLIP}}(\theta) = -\frac{1}{B} \sum_t \min\Big(r_t(\theta)\, \hat{A}_t,\ \ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, \hat{A}_t \Big)$$

    - **Value loss:**

$$\mathcal{L}^{\text{V}}(w) = \frac{1}{B} \sum_t \big(V_w(s_t) - \hat{R}_t\big)^2$$

    - **Entropy bonus:** $\mathcal{L}^{\text{H}}(\theta) = -\dfrac{1}{B} \sum_t H(\pi_\theta(\cdot \mid s_t))$
    - **Total loss:** $\mathcal{L}(\theta, w) = \mathcal{L}^{\text{CLIP}}(\theta) + c_v\, \mathcal{L}^{\text{V}}(w) + \beta\, \mathcal{L}^{\text{H}}(\theta)$
    - Gradient step: $\theta \leftarrow \theta - \alpha_\theta\, \nabla_\theta\, \mathcal{L}, \ \ w \leftarrow w - \alpha_w\, \nabla_w\, \mathcal{L}$

3. Return π_θ

✦ ✦ ✦

Why the clip works — intuition:

- If $\hat{A}_t > 0$ (action was better than average): we want to **increase** $\pi_\theta(a_t \mid s_t)$. The clip caps the surrogate at $r_t = 1 + \epsilon \Rightarrow$ no incentive to push the ratio further $\Rightarrow$ gradient vanishes once $r_t > 1 + \epsilon$.
- If $\hat{A}_t < 0$ (action was worse): we want to **decrease** $\pi_\theta(a_t \mid s_t)$. The clip caps it at $r_t = 1 - \epsilon$ $\Rightarrow$ no incentive to push r_t below $1 - \epsilon$.
- The min ensures the clipping only kicks in when it would make the objective *easier* — i.e., it's a **lower bound** on the unclipped surrogate, so optimisation is conservative.

Notes:

- **On-policy** but reuses each batch for K epochs (a few — typically K = 4-10) $\Rightarrow$ much more sample-efficient than vanilla policy gradient / A2C without the instability of off-policy methods.
- **GAE(λ)** smoothly interpolates between TD(0) advantage ($\lambda = 0$, low variance, high bias) and Monte Carlo advantage ($\lambda = 1$, high variance, low bias). Typical $\lambda \in [0.9, 0.97]$.

- **No second-order optimisation** (unlike TRPO): just SGD/Adam on the clipped objective $\Rightarrow$ simple to implement, well-suited to GPUs.
- An older PPO variant uses an **adaptive KL penalty** $-\beta_{KL}\,\mathrm{KL}(\pi_{\theta_{\text{old}}}\|\pi_\theta)$ instead of the clip; the clip variant is now standard.
- **Works for both discrete and continuous actions** (Gaussian policy for continuous) — one of the few RL algorithms that generalises across action spaces with minimal tuning.
- In practice, **PPO is the default first choice** for new problems: robust, well-understood, strong baselines (used in OpenAI Five, ChatGPT RLHF, etc.).

Additional Known Engineering Tricks

- **GAE(λ)** *(Schulman et al., 2016)* — replace the n-step advantage shown in the pseudocode with $\hat{A}_t = \sum_{l\geq 0}(\gamma\lambda)^l\,\delta_{t+l}$, where $\delta_t = r_t + \gamma V(s'_t; w) - V(s_t; w)$. *Why it helps:* lower-variance advantage than Monte-Carlo returns, lower-bias than one-step TD; typical $\lambda \in [0.9, 0.97]$ is the empirical sweet spot. *Interaction:* the resulting $\hat{A}$ values are normalised next.

- **Advantage normalisation per minibatch** — $\hat{A} \leftarrow (\hat{A} - \mu_{\hat{A}})/(\sigma_{\hat{A}} + 10^{-8})$. *Why it helps:* the loss is scale-invariant in advantage units, so the *same* clip range $\epsilon = 0.2$ works across all environments. *Interaction:* must be done *after* advantage computation but *before* the SGD epochs, so the same mean/std is reused within each batch.

- **Value-loss clipping** — clip $V_w(s_t)$ around the old value: $V^{\text{clip}} = V_{\text{old}} + \text{clip}(V_w - V_{\text{old}}, -\epsilon, +\epsilon)$, then take $\max((V_w - R_t)^2, (V^{\text{clip}} - R_t)^2)$. *Why it helps:* mirrors the policy clip on the critic so a single bad batch can't drag V wildly; controversial — *(Andrychowicz et al., 2021)* found ablating it harmless on most tasks.

- **Global-norm gradient clipping, threshold** ~ 0.5 — clip $\|\nabla(\theta, w)\|_2$ before each Adam step. *Why it helps:* the multi-epoch updates over a single batch can accumulate large gradients on the last epoch when the ratio r_t moves furthest; clipping caps the per-step damage.

- **Orthogonal weight initialisation, scaled per-head** — orthogonal init with gain $\sqrt{2}$ on hidden layers, **gain 0.01 on the policy logits head**, gain 1.0 on the value head. *Why it helps:* the tiny gain on the policy head ensures the initial policy is nearly uniform $\Rightarrow$ high initial entropy $\Rightarrow$ broad initial exploration. This is one of the well-known *implementation details* that separates a working PPO from a non-working one (see *Engstrom et al., 2020* and *Andrychowicz et al., 2021*).

- **Linear learning-rate annealing** — $\alpha_t = \alpha_0 \cdot (1 - t/T)$. *Why it helps:* the same recipe as supervised learning — large steps early, fine steps late; reduces the risk of late-training instability when the policy is near-converged.

- **Observation / reward normalisation with running stats** — maintain running (μ, σ) over all parallel envs, feed $(s-\mu)/\sigma$ to the network, and divide rewards by the running std of discounted returns. *Why it helps:* makes a single hyperparameter set work across tasks with completely different observation and reward scales — a recurring theme; PPO without these is the canonical example of brittle deep RL.

- **Early stopping by KL divergence** — abort the K-epoch loop if $\mathbb{E}_t[\mathrm{KL}(\pi_{\theta_{\text{old}}}\|\pi_\theta)] > 1.5\,\delta_{\text{target}}$. *Why it helps:* the clip alone is not enough when the advantage sign is mixed; an explicit KL target gives a hard guard against destructive updates. *Interaction:* combines naturally with the clip — the clip handles the typical case and KL early-stop catches the rare runaway minibatch.

Algorithm 4.7 PPO with clipped surrogate objective — Raw Pseudocode (without additional tricks) *[Schulman et al., 2017]*

Here is the pseudocode for the PPO core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:

Input: α_θ, α_w, γ, clip ϵ, epochs K, minibatch size B, workers K_{w}, rollout length n, entropy coefficient β, value coefficient c_v

Initialize policy parameters θ and value parameters w

repeat until convergence

$\theta_{\mathrm{old}} \leftarrow \theta$

for each worker $k = 1, \dots, K_{\mathrm{w}}$ **do**

Collect n steps using $\pi_{\theta_{\mathrm{old}}}$: (s_i, a_i, r_i, s'_i) for $i = 0, \dots, n-1$

end for

// Bootstrapped n-step return and advantage

$R \leftarrow 0$ if s'_{n-1} terminal else $V(s'_{n-1};\, w)$

for $i = n-1$ **down to** 0 **do**

$R \leftarrow r_i + \gamma\, R$

$R_i \leftarrow R; \quad \hat{A}_i \leftarrow R_i - V(s_i;\, w)$

end for

// Multi-epoch minibatch updates

for epoch $= 1, 2, \dots, K$ **do** multi-epoch reuse of one rollout

for each minibatch of size B **do**

$$r_t(\theta) \leftarrow \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}$$

$$\mathcal{L}^{\mathrm{CLIP}} \leftarrow \mathbb{E}_t\big[\, \min\!\big(r_t(\theta)\, \hat{A}_t,\ \ \mathrm{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\, \hat{A}_t\big)\, \big]$$

$$\mathcal{L}^{V} \leftarrow \mathbb{E}_t\big[\, (R_t - V(s_t;\, w))^2\, \big]$$

$$\mathcal{L}^{H} \leftarrow \mathbb{E}_t\big[\, \mathcal{H}(\pi_\theta(\cdot \mid s_t))\, \big]$$

$$\mathcal{L} \leftarrow -\,\mathcal{L}^{\mathrm{CLIP}} + c_v\, \mathcal{L}^{V} - \beta\, \mathcal{L}^{H}$$

$$\theta \leftarrow \theta - \alpha_\theta\, \nabla_\theta\, \mathcal{L}; \quad w \leftarrow w - \alpha_w\, \nabla_w\, \mathcal{L}$$

end for

end for

end repeat

Algorithm 4.7 PPO with clipped surrogate objective — Pseudocode with Additions (Engineering Tricks) *[Schulman et al., 2017]*

Here is the pseudocode for the PPO algorithm with also employing all of the additions and engineering tricks mentioned above:

Input: ✦ initial lr α_0 ✦ (Linear learning-rate annealing), α_w, γ, clip ϵ, epochs K, minibatch size B, workers K_{w}, rollout length n, entropy coefficient β, value coefficient c_v, ✦ GAE λ ✦ (GAE(λ)), ✦ gradient clip $c = 0.5$ ✦ (Global-norm gradient clipping, threshold ~ 0.5), ✦ KL target δ_{KL} ✦ (Early stopping by KL divergence)

✦ Initialise Actor and Critic with orthogonal weights (gain $\sqrt{2}$ hidden, gain 0.01 policy logits head, gain 1.0 value head) ✦ (Orthogonal weight initialisation, scaled per-head)

✦ Initialise running observation stats (μ_s, σ_s) and reward normaliser ✦ (Observation / reward normalisation with running stats)
repeat until convergence
  ✦ $\alpha_t \leftarrow \alpha_0 \cdot (1 - t/T)$ *(linear lr annealing)* ✦ (Linear learning-rate annealing)
  $\theta_{\text{old}} \leftarrow \theta$
  for each worker $k = 1, \dots, K_{\text{w}}$ **do**
    ✦ Normalise observations: $s \leftarrow (s - \mu_s)/\sigma_s$ ✦ (Observation / reward normalisation with running stats)
    Collect n steps using $\pi_{\theta_{\text{old}}}$: (s_i, a_i, r_i, s'_i) for $i = 0, \dots, n-1$
  end for
  ✦ *// Compute GAE advantages* ✦ (GAE(λ))
  $\delta_i \leftarrow r_i + \gamma\, V(s'_i;\, w) - V(s_i;\, w)$
  ✦ $\hat{A}_i \leftarrow \sum_{l \geq 0} (\gamma\lambda)^l\, \delta_{i+l}$ ✦ (GAE(λ))
  $R_i \leftarrow \hat{A}_i + V(s_i;\, w)$
  ✦ $\hat{A} \leftarrow (\hat{A} - \mu_{\hat{A}})/(\sigma_{\hat{A}} + 10^{-8})$ *(advantage normalisation)* ✦ (Advantage normalisation per minibatch)
  // Multi-epoch minibatch updates
  for epoch $= 1, 2, \dots, K$ **do**
    ✦ **if** $\mathbb{E}_t[\mathrm{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta)] > 1.5\, \delta_{\text{KL}}$ **then break** *(early stopping)* ✦ (Early stopping by KL divergence)
    for each minibatch of size B **do**
      $r_t(\theta) \leftarrow \dfrac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$
      $\mathcal{L}^{\text{CLIP}} \leftarrow \mathbb{E}_t\big[\, \min\!\big(r_t(\theta)\, \hat{A}_t,\;\; \mathrm{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\, \hat{A}_t\big) \,\big]$
      ✦ $V^{\text{clip}} \leftarrow V_{\text{old}} + \mathrm{clip}(V_w - V_{\text{old}},\, -\epsilon,\, +\epsilon)$ ✦ (Value-loss clipping)
      ✦ $\mathcal{L}^V \leftarrow \mathbb{E}_t\big[\, \max\!\big((V_w - R_t)^2,\; (V^{\text{clip}} - R_t)^2\big) \,\big]$ ✦ (Value-loss clipping)
      $\mathcal{L}^H \leftarrow \mathbb{E}_t\big[\, \mathcal{H}(\pi_\theta(\cdot \mid s_t)) \,\big]$
      $\mathcal{L} \leftarrow -\mathcal{L}^{\text{CLIP}} + c_v\, \mathcal{L}^V - \beta\, \mathcal{L}^H$
      ✦ **if** $\|\nabla \mathcal{L}\|_2 > c$ **then** rescale to norm c ✦ (Global-norm gradient clipping, threshold ~ 0.5)
      ✦ $\theta \leftarrow \theta - \alpha_t\, \nabla_\theta\, \mathcal{L}; \quad w \leftarrow w - \alpha_t\, \nabla_w\, \mathcal{L}$ *(annealed lr)* ✦ (Linear learning-rate annealing)
    end for
  end for
end repeat

✦✦✦

✦ ✦ ✦

1.8 Offline RL (Batch RL)

✦ ✦ ✦

1.8.1 CQL (Conservative Q-Learning)

Notation: - $\mathcal{D}$ — fixed offline dataset of transitions (no new environment interactions during training) - λ — CQL regularisation weight; controls conservatism strength - $\mathcal{L}_{\text{CQL}}(\theta)$ — conservative penalty: pushes Q down for all actions (via logsumexp), up for actions observed in $\mathcal{D}$ - π_β — behavior policy: the (unknown) policy that originally collected $\mathcal{D}$

Same as DQN+ER above: - θ^- — frozen target network weights (a periodic hard copy of θ; never updated by gradients) - $\widehat{Q}(s, a; \theta^-)$ — target network (separate from the online network Q) - $\mathcal{L}(\theta)$ — loss function (squared Bellman error) - B — minibatch size (transitions sampled per gradient update) - C — target update frequency (gradient steps between hard copies $\theta^- \leftarrow \theta$)

Introduction: Offline (batch) RL — learns entirely from a ***fixed dataset*** $\mathcal{D}$ with no environment interaction during training. Builds on DQN+ER but adds a ***conservative regulariser*** to combat overestimation of ***out-of-distribution (OOD)*** actions. *Why offline RL is hard:* in online Q-learning, overestimation of Q for unseen actions is self-correcting (the agent tries the action, gets a bad reward, Q is corrected); offline there is no such correction, and the $\max_{a'}$ in the Bellman target latches onto inflated OOD Q-values ⇒ divergence. *CQL's fix:* push Q down for *all* actions (via a $\log \sum_a \exp Q$ term) while pushing it back up for actions actually observed in $\mathcal{D}$ — so OOD Q-values become *lower bounds* of their true values and the greedy policy avoids them.

Algorithm:

Input: fixed dataset $\mathcal{D} = \{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$, discount γ, learning rate α, CQL weight λ, minibatch size B, target update freq C, training iterations n

1. Initialize **online network** $Q(s, a; \theta)$; **target network** $\widehat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$

   Initialize step counter $t \leftarrow 0$

2. **For** iteration $= 1, 2, \ldots, n$ *(training loop — no environment interaction)*:

   - Sample minibatch of B transitions $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$ from $\mathcal{D}$
   - **Compute standard Bellman targets:**

$$y_j = \begin{cases} r_j & \text{if done}_j \\ r_j + \gamma \max_{a'} \widehat{Q}(s'_j, a';\, \theta^-) & \text{otherwise} \end{cases}$$

   - **Compute CQL conservative penalty:**

$$\mathcal{L}_{\text{CQL}}(\theta) = \underbrace{\mathbb{E}_{s \sim \mathcal{D}}\left[\log \sum_a \exp Q(s, a; \theta)\right]}_{\text{push down } Q \text{ for all actions (soft-max)}} - \underbrace{\mathbb{E}_{(s,a) \sim \mathcal{D}}[Q(s, a; \theta)]}_{\text{push up } Q \text{ for dataset actions}}$$

- **Compute total loss:**

$$\mathcal{L}(\theta) = \underbrace{\lambda\, \mathcal{L}_{\text{CQL}}(\theta)}_{\text{conservative regulariser}} + \underbrace{\frac{1}{B}\sum_j \big(y_j - Q(s_j, a_j; \theta)\big)^2}_{\text{standard TD loss}}$$

- **Gradient update:**

$$\theta \leftarrow \theta - \alpha\, \nabla_\theta\, \mathcal{L}(\theta)$$

- $t \leftarrow t + 1$; if $t \bmod C = 0$: $\theta^- \leftarrow \theta$ *(sync target network)*

3. Return $Q(\cdot, \cdot; \theta)$; derive $\pi(s) = \arg\max_a Q(s, a; \theta)$

✦ ✦ ✦

Why offline RL is hard — and how CQL helps:

- In standard (online) Q-learning, overestimation of Q for unseen actions is self-correcting: the agent tries the action, gets a bad reward, and Q is corrected. In **offline RL there is no such correction** — the agent never interacts with the environment.
- Naive application of DQN to a fixed dataset leads to severe **overestimation for OOD actions** (actions not well-represented in $\mathcal{D}$). The $\max_{a'}$ in the Bellman target selects these overestimated actions ⇒ divergence.
- **CQL's fix:** the conservative penalty $\mathcal{L}_{\text{CQL}}$ systematically **pushes Q-values down** for all state-action pairs (via the logsumexp term) while **pushing them back up** for actions actually observed in the dataset. Net effect: Q values for OOD actions are *lower bounds* of their true values ⇒ the policy avoids OOD actions.
- λ controls the strength of conservatism: larger λ ⇒ more conservative (safer but potentially suboptimal); smaller λ ⇒ closer to standard Q-learning (riskier).

Notes:

- **No exploration at all** — the entire algorithm runs on the fixed dataset $\mathcal{D}$.
- $\mathcal{D}$ can come from any source: expert demonstrations, a previous policy, random exploration, or a mix (any behavior policy π_β).
- The logsumexp term $\log \sum_a \exp Q(s, a; \theta)$ is a smooth approximation of $\max_a Q(s, a; \theta)$.
- CQL can also be combined with actor-critic methods (CQL-SAC variant for continuous actions).
- Other offline RL approaches include **BCQ** (constrain policy to dataset support), **TD3+BC** (add behavioral cloning regularisation), and **IQL** (avoid querying OOD actions entirely via expectile regression).

Additional Known Engineering Tricks

- **Lagrangian dual for automatic λ** *(CQL(H) variant)* — define a constraint $\mathbb{E}_{s\sim\mathcal{D}}[\log \sum_a \exp Q(s, a) - \mathbb{E}_{a\sim\pi_\beta} Q(s, a)] \le \tau$ and update λ by dual gradient ascent on the violation. *Why it helps:* removes the single most sensitive hyperparameter — λ now auto-adapts to the dataset's coverage; too-conservative when off-support actions get high Q, relaxes once they don't.
- **CQL-SAC: combine with twin critics + entropy-regularised actor** — apply the CQL penalty to *both* critics of a SAC base, and let the actor be the entropy-regularised SAC actor.

Why it helps: offline RL needs continuous-action support; the SAC base brings reparam + min-of-twin Qs, the CQL term brings out-of-distribution conservatism. *Interaction:* this is the de-facto practical variant for continuous-control offline RL benchmarks (D4RL).

- **Importance-weighted action sampling for the logsumexp** — approximate $\log \sum_a \exp Q(s,a)$ on continuous actions by sampling actions from three sources (uniform, current policy, dataset policy) and combining with importance weights. *Why it helps:* the logsumexp over a continuous action space is intractable; sampling from a mixture covers the relevant support without bias. *Interaction:* the *uniform* branch is what pushes down OOD actions; the *policy* branch focuses computation on the actions actually selected; the *dataset* branch keeps things grounded in observed data.
- **Behavioural cloning regulariser** — add $-\alpha_{\text{BC}}\, \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log \pi(a \mid s)]$ to the actor loss. *Why it helps:* explicitly keeps the learned policy close to π_β on the dataset's support, complementing CQL's Q-side conservatism; widely used in TD3+BC. *Interaction:* with strong CQL the BC term is redundant; with weak CQL it's a useful safety net.
- **State (and action) normalisation** — apply running-stats normalisation to states and continuous actions before they enter the network. *Why it helps:* offline RL has no exploration to compensate for badly-scaled inputs, so the network must extract every bit of signal from the fixed $\mathcal{D}$; whitening makes that signal scale-invariant.
- **Layer normalisation throughout** — between every hidden layer of both Q-networks and the actor. *Why it helps:* offline training does many gradient steps per data point and is prone to representational collapse; LayerNorm dramatically stabilises this regime — a common ablation finding in modern offline RL papers (e.g., *Ball et al., 2023*).
- **Min-of-twin Bellman target** — even before CQL, take $\min(\hat{Q}_{\theta_1^-}, \hat{Q}_{\theta_2^-})$ in the standard TD target. *Why it helps:* the same overestimation fix that TD3 brings to online RL is *especially* important offline, because the agent never gets to test its overestimates against the real environment.

Algorithm 4.8 CQL (conservative Q-learning, offline) — Raw Pseudocode (without additional tricks) *[Kumar et al., 2020]*

Here is the pseudocode for the CQL core (base) algorithm without any further additions and engineering tricks (also mentioned above) added:
Input: fixed dataset $\mathcal{D} = \{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$, discount γ, learning rate α, CQL weight λ, minibatch size B, target update freq C, training iterations n
Initialize online network $Q(s,a;\theta)$; target network $\hat{Q}(s,a;\theta^-)$ with $\theta^- \leftarrow \theta$
for $t = 1, 2, \ldots, n$ **do**
Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$
// Standard TD target (data only — no environment interaction)
$y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)\, \max_{a'} \hat{Q}(s'_j, a'; \theta^-)$
$\mathcal{L}_{\text{TD}} \leftarrow \dfrac{1}{B} \sum_j \big(y_j - Q(s_j, a_j; \theta) \big)^2$
// Conservative regulariser: push down OOD actions, push up data actions
$\mathcal{L}_{\text{CQL}} \leftarrow \dfrac{1}{B} \sum_j \Big[\log \sum_a \exp Q(s_j, a; \theta) \;-\; Q(s_j, a_j; \theta) \Big]$

$\mathcal{L}(\theta) \leftarrow \mathcal{L}_{\text{TD}} + \lambda\,\mathcal{L}_{\text{CQL}}$
$\theta \leftarrow \theta - \alpha\,\nabla_\theta\,\mathcal{L}(\theta)$
if $t \bmod C = 0$ **then** $\theta^- \leftarrow \theta$
end for

Algorithm 4.8 CQL (conservative Q-learning, offline) — Pseudocode with Additions (Engineering Tricks) *[Kumar et al., 2020]*

Here is the pseudocode for the CQL algorithm with also employing all of the additions and engineering tricks mentioned above:
Input: fixed dataset $\mathcal{D} = \{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$, discount γ, lr α, ✦ initial λ, constraint τ_{CQL} ✦ (Lagrangian dual for automatic λ), minibatch B, target update freq C, iterations n, ✦ BC weight α_{BC} ✦ (Behavioural cloning regulariser), ✦ entropy temperature α_{ent} ✦ (CQL-SAC: combine with twin critics + entropy-regularised actor)
✦ Initialise twin online critics $Q_{\theta_1}, Q_{\theta_2}$ and twin target critics $\hat{Q}_{\theta_1^-}, \hat{Q}_{\theta_2^-}$ with LayerNorm ✦ (CQL-SAC: combine with twin critics + entropy-regularised actor / Layer normalisation throughout)
✦ Initialise entropy-regularised stochastic actor π_ϕ ✦ (CQL-SAC: combine with twin critics + entropy-regularised actor)
✦ Initialise learnable $\log\lambda$ ✦ (Lagrangian dual for automatic λ)
✦ Compute running stats (μ_s, σ_s) from $\mathcal{D}$; normalise all states and actions ✦ (State (and action) normalisation)
for $t = 1, 2, \ldots, n$ **do**
Sample minibatch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}_{j=1}^{B} \sim \mathcal{D}$
// TD target (CQL-SAC: twin-min + entropy)
Sample $\tilde{a}' \sim \pi_\phi(\cdot \mid s'_j)$
✦ $y_j \leftarrow r_j + \gamma\,(1 - \text{done}_j)[\min_{i=1,2} \hat{Q}_{\theta_i^-}(s'_j, \tilde{a}') - \alpha_{\text{ent}} \log \pi_\phi(\tilde{a}' \mid s'_j)]$ *(min-of-twin + SAC entropy)* ✦ (Min-of-twin Bellman target / CQL-SAC: combine with twin critics + entropy-regularised actor)
$\mathcal{L}_{\text{TD}}^{(i)} \leftarrow \frac{1}{B}\sum_j (y_j - Q_{\theta_i}(s_j, a_j))^2$
// Conservative regulariser
✦ Sample actions from three sources: $a^{\text{unif}} \sim \text{Uniform}$, $a^\pi \sim \pi_\phi(\cdot \mid s_j)$, a^β from dataset ✦ (Importance-weighted action sampling for the logsumexp)
✦ $\mathcal{L}_{\text{CQL}}^{(i)} \leftarrow \frac{1}{B}\sum_j [\log \sum_{\tilde{a}} \exp Q_{\theta_i}(s_j, \tilde{a}) \,/\, w(\tilde{a}) \;-\; Q_{\theta_i}(s_j, a_j)]$ *(IS-weighted logsumexp)* ✦ (Importance-weighted action sampling for the logsumexp)
✦ $\lambda \leftarrow \exp(\log\lambda)$; $\log\lambda \leftarrow \log\lambda + \alpha_\lambda\,(\mathcal{L}_{\text{CQL}} - \tau_{\text{CQL}})$ *(dual update)* ✦ (Lagrangian dual for automatic λ)
for $i = 1, 2$ **do** $\theta_i \leftarrow \theta_i - \alpha\,\nabla_{\theta_i}(\mathcal{L}_{\text{TD}}^{(i)} + \lambda\,\mathcal{L}_{\text{CQL}}^{(i)})$ **end for**
// Actor update (SAC-style + BC)
Sample $\tilde{a}_j \sim \pi_\phi(\cdot \mid s_j)$
✦ $\mathcal{L}_\pi \leftarrow \frac{1}{B}\sum_j [\alpha_{\text{ent}} \log \pi_\phi(\tilde{a}_j \mid s_j) - \min_i Q_{\theta_i}(s_j, \tilde{a}_j)] - \alpha_{\text{BC}}\,\frac{1}{B}\sum_j \log \pi_\phi(a_j \mid s_j)$ *(+ BC regulariser)* ✦ (CQL-SAC: combine with twin critics + entropy-regularised actor / Behavioural cloning regulariser)

$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}_\pi$
if $t \bmod C = 0$ **then** $\theta_i^- \leftarrow \theta_i$ for $i = 1, 2$
end for

✦✦✦